

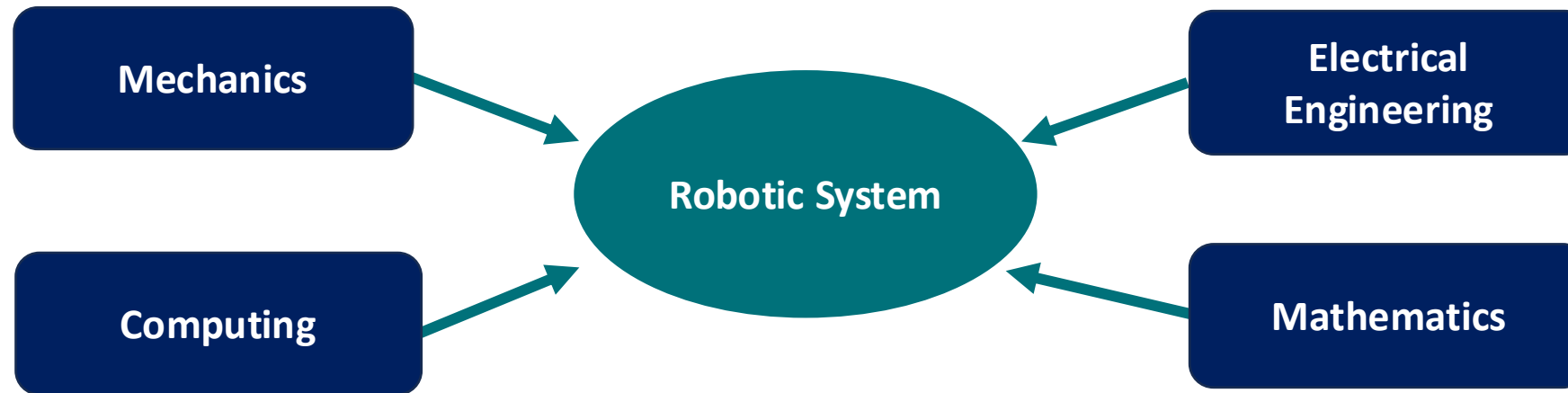
ROS2

Mathéo Delannoy
matheo.delannoy@edu.ece.fr



Motivation (1 / 3)

- A robotic system is very complex and typically involves many people working on a project, each bringing specialized knowledge in areas such as:



- In order to be productive in the development of robotic systems, it is therefore essential to **reuse existing software components** and collaborate efficiently within multidisciplinary teams.

Motivation (2 / 3)

- In software projects, platforms like Linux exist because of investments from developers and companies that prefer not to develop entire operating systems from scratch. For robotics, the first major collaborative robotics framework is **ROS (Robot Operating System)**.
- ROS initially started as a personal project in **2006 by Keenan Wyrobek and Eric Berger at Stanford University**, designed to eliminate the "reinventing the wheel" problem that was prevalent in robotics development, as demonstrated in their original pitch presentation.



Slide extract from Eric and Keenan pitch deck.

Motivation (3 / 3)

- **No developer starts programming a robot from scratch** because robotic software stacks are extremely complex. Implementing low-level hardware drivers requires extensive expertise and time.



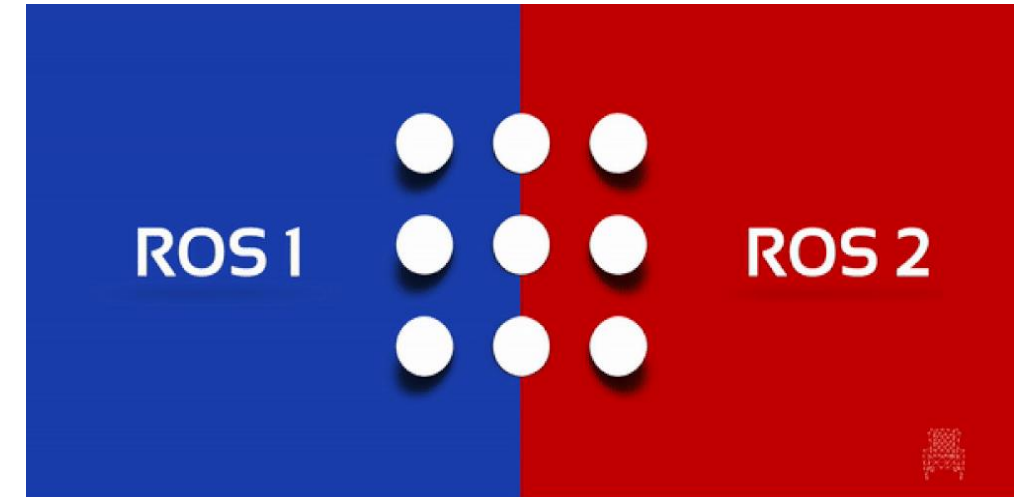
For instance, the ROS2 driver package for v4l2-compatible cameras (such as Raspberry Pi cameras) contains over 10,000 lines of code to handle camera initialization, configuration, and data streaming.



- A middleware framework like ROS2 enables significant time savings and allows students to implement large-scale robotic applications by providing:
 - Development **APIs** (Application Programming Interfaces) for reusing and integrating existing solutions.
 - **Inter-process communication** mechanisms between software components and hardware systems.
 - **Hardware abstraction layers** that simplify access to sensors and actuators.
 - Core robotics services including path planning, SLAM (Simultaneous Localization and Mapping), and navigation algorithms.
 - Comprehensive development, debugging, and testing tools for robotic applications (Rvizz, Gazebo, etc.)

Why ROS2 ?

- Inter components communication ;
- Integrated development including debugging and viualization platforms (**Rviz**) ;
- Large ecosystem of available libraries and packages ;
- Active community that continues evolving and improving the framework ;
- Code reusability across different robotic platfoms and apps ;
- Rapid testing with **Gazebo** simulator (or recently Nvidia Isaac SIM), and simple way to record and playback sensor data with **rosbags**.



ROS2 (development starting in 2014) is architecturally similar to ROS1 but it is built on **DDS** (Data Distribution Service) middleware. This represents a significant enhancement designed to transition from **laboratory development environments to real-world application**.

Alternatives to ROS

- ROS, due to its large community and status as the most adopted robotic middleware, is often considered mandatory for robotic dev. But for specific performance requirement or to **avoid being lock-in to the ROS ecosystem**, alternative solutions exist and might be better suited depending on the project.

Robotic Middleware	Open-source	Windows	Linux	Composite Component	Robot Software architecture independant	Graphical IDE	Simulation Environment	Real-time
ROS2	YES	NO	YES	NO	YES	NO	YES, RVIZ, GAZEBO	MID
RT-Middleware	YES	YES	YES	YES	YES	YES	YES, OpenHRP, choreonoid	YES
OPRoS	YES	YES	YES	YES	YES	YES	YES, OPRoS Simulator	MID
Orocos	YES	YES	YES	NO	YES	NO	NO	YES

Table of contents

1. Prerequisites and basic knowledge

p. 8

2. ROS2 Introduction

p. 12

3. Development environment setup

- Ubuntu Platform installation p. 20
- Remote development with SSH p. 25
- ROS2 Installation and Configuration p. 27
- Development tools and software setup p. 31

4. ROS2 Basics

- Nodes p. 37
- Topics and messages Communication p. 42
- Computation graph p. 44
- Others Communication (Services and Actions) p. 45

5. Workspace and Package development

- Workspace creation and build systems p. 49
- Package creation and structure p. 53
- Launch File p. 61
- Plugin creation and usage p. 65

6. Robot modeling and simulation

- TF2 p. 71
- URDF p. 74
- Xacro p. 83
- Visualization tools (Foxglove / RViz) p. 87

7. micro-ROS

- Micro-ROS introduction p. 95
- Demonstrations p. 100
- Perspectives p. 108

8. Autonomous vehicles systems

- Sensors and data processing p. 110
- Perception p. 111
- Localisation and SLAM p. 112
- Planning p. 113
- Control p. 114

9. Advanced Resources

- Conclusion p. 116
- CLI Cheat Sheet p. 117
- Bibliography p. 118

1 Prerequisites and basic knowledge

Prerequisites and basic knowledge

- ROS2 support on [Windows](#) and [macOS](#) is still being developed. Unless you are an experienced software developer, it is recommended to work with Linux. Also, **mastering Linux is crucial for you as it is often used in development.**
- Ubuntu is the primary Linux distribution used for ROS2 dev. To learn how to use Linux, you can follow the course proposed for free by The Construct : [Linux for robotics](#)
Even for advanced users review Linux filesystem: [Linux Directories Explained in 100 Seconds](#)
- You will frequently use version control systems to collaborate in teams, manage project versions, clone existing repositories and fork projects to customize them for your specific needs. Basic knowledge in Git is important. To learn it, you can follow the tutorial available on Boostcamp: [Toolbox → GITHUB](#)



git



Prerequisites and basic knowledge

- ROS2 applications can be developed primarily in Python or C++. Mixed-language projects are fully supported since nodes communicate through ROS message. This is why you will often work in projects that use both. You need to know basis of Python, C++ and understand OOP (Object-Oriented Programming). The choice between C++/Python for a project depends on the requirements :

Python	C++
Easier and faster to build a prototype because the language does a lot of work	Code runs faster and is more optimized
Short programs for complex ideas	Harder than Python to master and become an expert
Does not need to compile	Needs to compile (which is better in reality), you can catch a lot of errors before having them in run time
Often used for IA	Necessary for real time code
Easier to integrate it with web servicess based on Django	Language mainly used in the robotics industry
Learn with : Python tutorial by W3 schools	Learn with : learncpp.com

Prerequisites and basic knowledge

- Annex for additional knowledge to learn:
 - Compiling
 - Basics of **computer science**
 - Basics of **electronics**
 - Basics of **physics**

2 ROS2 Introduction

ROS2 Introduction (1 / 6)

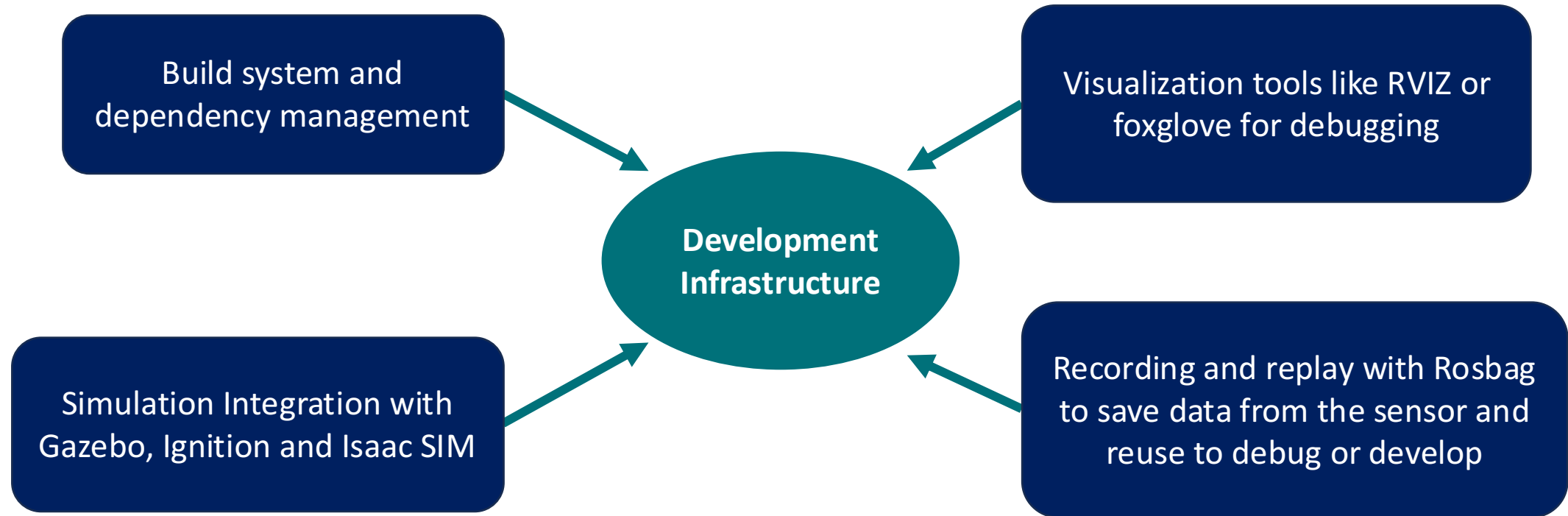
Middleware = Layer of Software between Operating system and user application.

- ROS stands for **Robot Operating System**, however it is **not an actual OS**!
- It can be described as a middleware that is open-source. The core components are :
 - **Communication system** based on **DDS** with a publish subscribe architecture for inter-process communication.
 - **Software libraries** and tools to build robot applications.
 - **Hardware abstraction** with software drivers to easily work with various HW components.
 - Open-source algorithms.
- The ecosystem is also important :
 - 2 programming languages are officially supported: **C++ and Python** (we will also use **XML** and **XACRO** for configuration and robot description).
 - The community is very active; people contribute to the project and help.

Did you know that ROS stands for Robot Operating System but it is not an actual OS haha

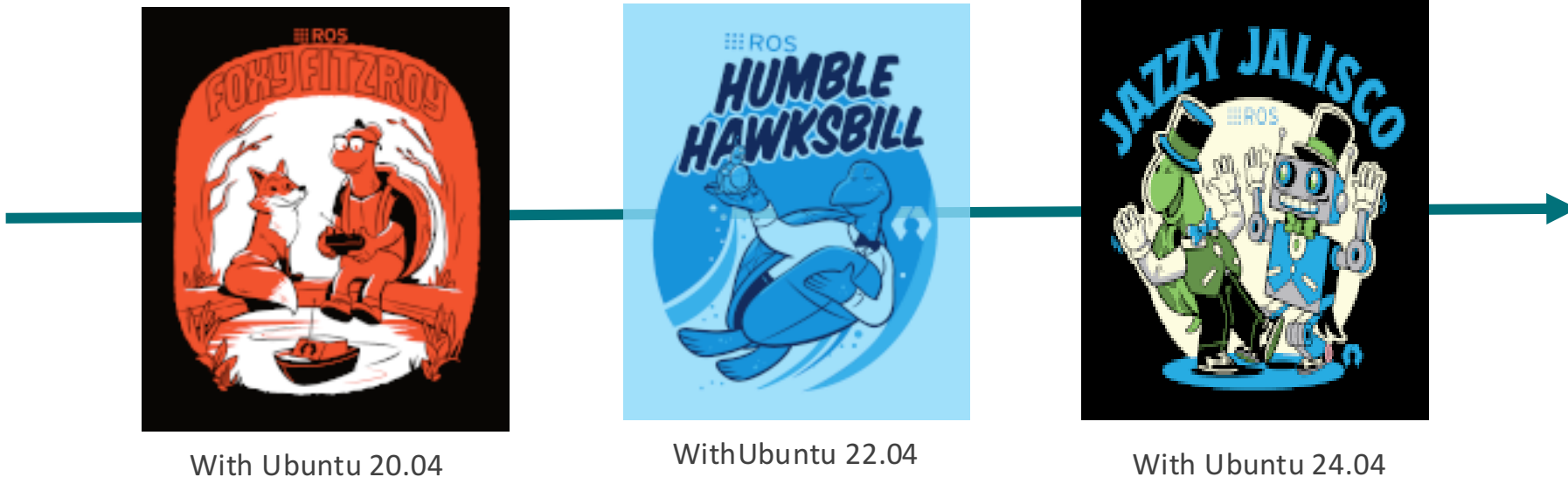


ROS2 Introduction (2 / 6)



ROS2 Introduction (3 / 6)

- ROS is organized into **distributions**, which are versioned sets of ROS packages. Each distribution ensures all packages work together on a specific OS, provides a stable codebase for developers and receives updates/patches during its support lifecycle.
- For this tutorial, we are using Ubuntu 22.04, which pairs with the [Humble Hawksbill distribution](#). It is a long term support release, maintained until May 2027, ensuring stability for large scale projects.



ROS2 Introduction (4 / 6)

- In ROS often, developers follow a distributed computing architecture that separates development and deployment environments.



Development Machine: With a Full OS and ROS2 installation with high computing power, for development, simulation, visualization and testing.



Embedded system: Lightweight ROS2 deployment on resource-constrained HW:

- Raspberry PI (Runs standard ROS2)
- Microcontroller (STM32, ESP32) will use **micro-ROS**.

ROS2 Introduction (5 / 6)

Dataflow

Provides an interface to the sensors or other components

The OS provides a **low-level driver** to interface with the HW

Data transmission between hardware and higher layers

Standard that enable data exchanges using a **publish-subscribe pattern**.
Control over QoS options

...

User packages and nodes to communicate with HW

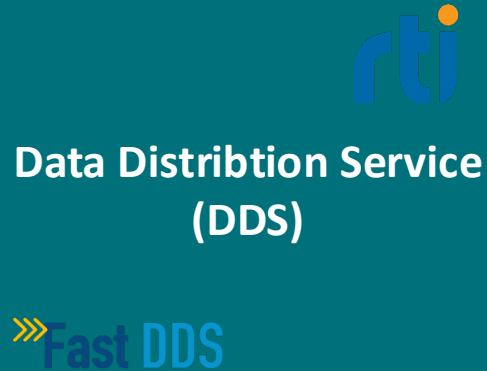


Hardware Layer



Operating system

Transport layer



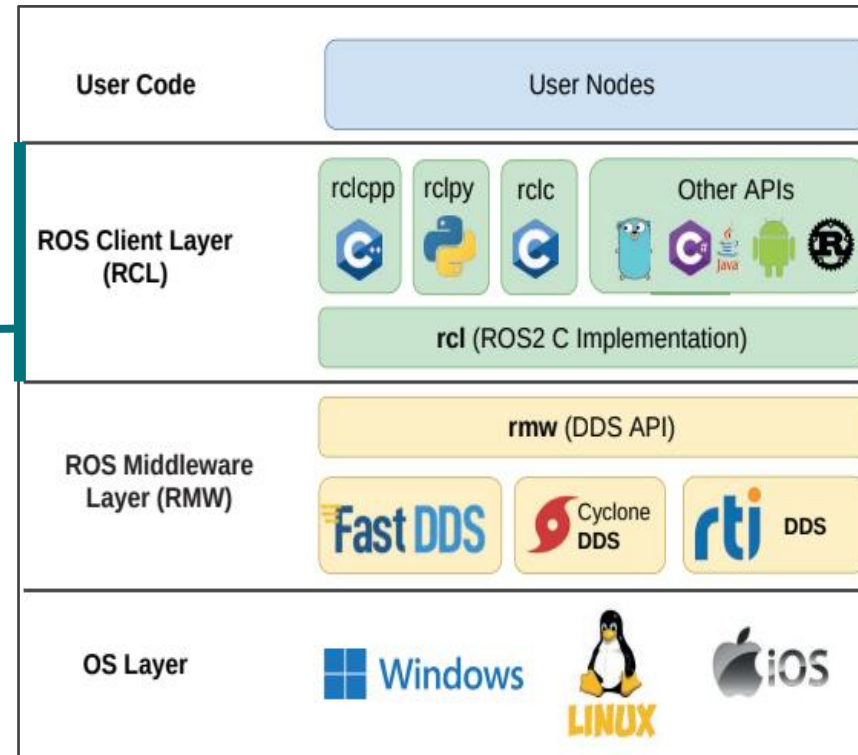
Data Distribution Service (DDS)

ROS layers

Application code

ROS2 Introduction (6 / 6)

- **RCL (ROS Client Library)** provides a high level interface for building and running robot applications using rmw. Allow developer to **use core ROS 2 functionalities** such as node, topic, services etc. **without having to worry about how RMW works.**
- **Rclcpp** is the **client Library C++ Wrapper**. Gives a set of classes, functions and features to develop, build and run projects using RCL.
- **Rclpy** – similar as rclpp but in python



- **RMW (ROS MiddleWare)** layer does the middleware hard work for developers.
- It ensures that the application code can **interact with the DDS**, by providing a communication infrastructure, without depending on its specific implementation details.
- Normally you **do not need to worry about how DDS works.**

3 Development Environment Setup

Ubuntu platform installation (1 / 5)

- As we are working with ROS2 Humble, the required OS is Ubuntu 22.04, you can download it to install on the link: <https://releases.ubuntu.com/jammy/>
- In this part we will see three installation approach depending on your hardware and requirements (another possibility not yet mentioned is using WSL):

Virtual Machine

Install Ubuntu 22.04 as a virtual machine on windows using VirtualBox.

- ✓ Very **easy** and safe, simple to remove.
- ✗ **Reduced performance**, limited hardware optimisation (small GPU acceleration).
- ✗ USB device access or network configuration can be problematic.

Dual Boot

Alongside your OS (windows or macOS) with a dedicated partition.

- ⚠ Make sure **you know what you are doing**, risk of data loss if not carefull.
- ✓ Native hardware performances, optimal for development.
- ✓ Direct hardware access (sensors).
- ✗ Complex and must reboot to switch between OS.

Windows WSL

Ubuntu platform installation (2 / 5)

Virtual Machine Installation

Ubuntu platform installation (3 / 5)

Dual Boot

Can follow a tutorial but **think before acting!**



(Make sure you have a 16 GB USB FLASH)

- ① Download Ubuntu Desktop iso image.
- ② Install Rufus to create bootable USB : [Rufus download](#).
- ③ On Rufus choose your USB drive, the right **iso image**, make sure to know your **computer partition scheme** and finally choose **BIOS or UEFI on target system**.
- ④ **Partition your disk** by allocating the space on your disk for ubuntu using the **disk management** (100 GB is great for large use).
- ⑤ Turn off your computer go on your BIOS, **disable the Secure boot** and select the right boot device. You will be able then to click on try or install Ubuntu.
- ⑥ It's done! Now each time you will start your computer, you will be able to choose on the GRUB on wich OS you want to be.

Ubuntu platform installation (4 / 5)

Windows WSL

Ubuntu platform installation (5 / 5)

Description	Bash command
List files and folders on the current directory	<code>ls [options] [directory]</code>
Change directory	<code>cd [directory]</code>
Create a new folder	<code>mkdir [directory]</code>
Delete a file	<code>rm [options] [file]</code>
Display the content of a file	<code>cat [file]</code>
Move a file or a folder	<code>mv [source] [destination]</code>
Copy a folder	<code>cp [source] [destination]</code>
Display the information about a command	<code>man [command]</code>
Change file permissions	<code>chmod [permissions] [file]</code>
Display information about active processes	<code>ps [options]</code>
Terminate a process	<code>kill [process_ID]</code>

Remote development with SSH (1 / 2)

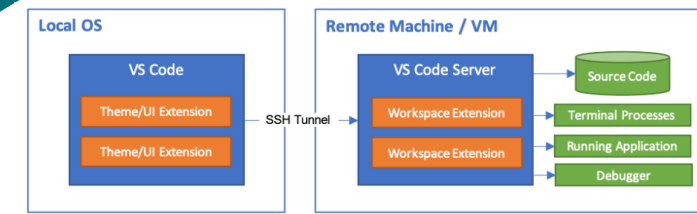
- As you may work with a raspberry or any other embedded computer, if you want to work remote easily with a nice interface you can use SSH. Before connecting make sure to execute the following command on remote:

```
sudo apt install openssh-server  
sudo systemctl enable ssh  
sudo systemctl start ssh  
sudo ufw allow ssh  
sudo ufw enable  
ifconfig #Will give the ip adress of the remote device
```

- You can now connect on your personal laptop, open a terminal and use the command (make sure to have both devices connected on the same network):

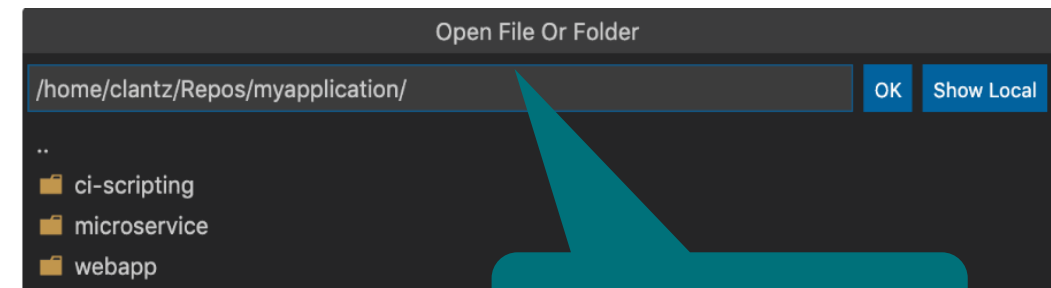
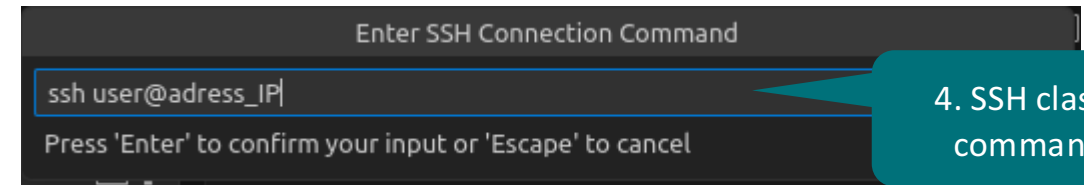
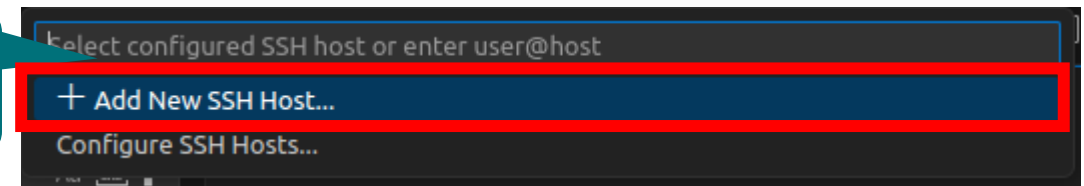
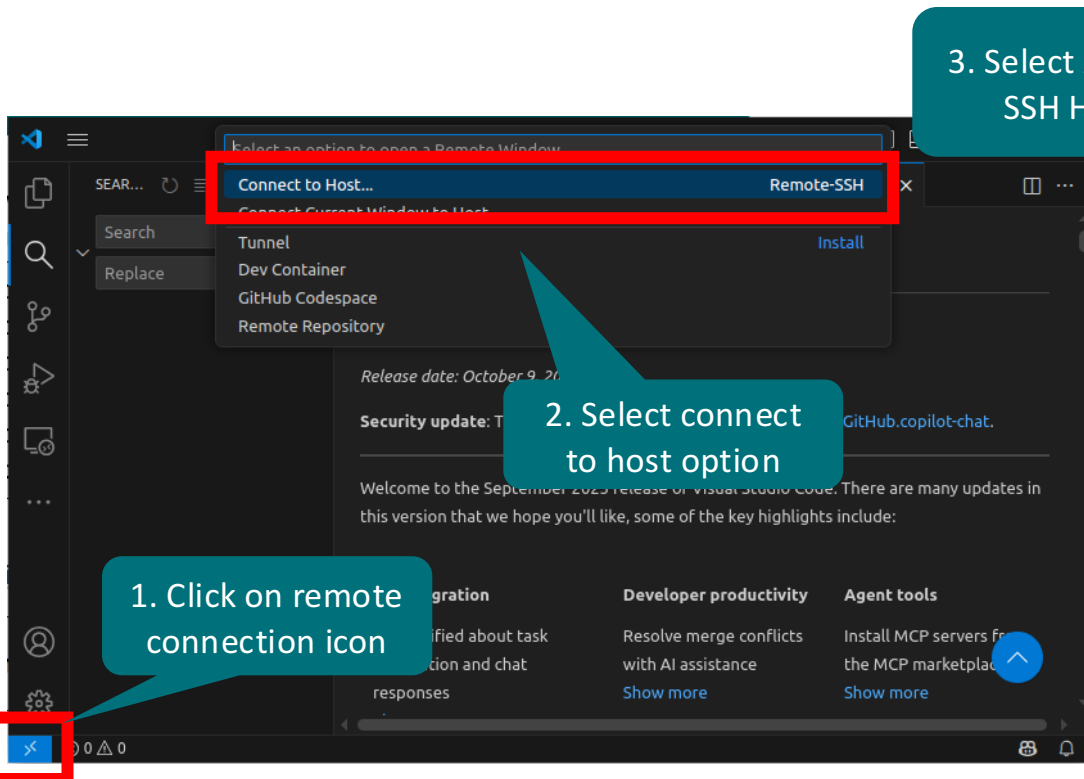
```
ssh username@IP_adress #Then enter your password
```

Remote development with SSH (2 / 2)



VS Code SSH extension

- With ROS2 workspace, you will manage a large amount of files, working in the terminal will be difficult, to avoid this we will use the visual studio code remote – ssh extension.



ROS2 Installation and Configuration (1 / 4)

- Before installing anything, you will need to make sure your **system locale supports UTF-8**.

```
locale #If UTF- does not appear run the command below
sudo apt update && sudo apt install locales
sudo locale-gen en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8 #As you wish
export LANG=en_US.UTF-8
locale #Normally it appears now
```

- As Ubuntu programs are stored in repositories, for installing ROS we need to **enable Ubuntu Universe repository**.

```
sudo apt install software-properties-common
sudo add-apt-repository universe
```

ROS2 Installation and Configuration (2 / 4)

- The next step is to add the ROS2 repository to the system, by authorizing the public GPG key provided by ROS and then add it to the **source list**.

```
sudo apt update && sudo apt install curl gnupg lsb-release

sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o
/usr/share/keyrings/ros-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-
keyring.gpg] http://packages.ros.org/ros2/ubuntu $(source /etc/os-release && echo
$UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

- Install ROS2 packages depending on your machine.

```
sudo apt update #Update your apt repo caches
sudo apt upgrade
sudo apt install ros-humble-desktop #Classic Desktop installation with a lot of included
tools
#sudo apt install rost-humble-ros-base #Minimum installation no GUI tools (Bare bones)
interesting (decomment if use)
sudo apt install ros-dev-tools #Development tools like compilers and other tools
```


ROS2 Installation and Configuration (3 / 4)

- We will now source the setup, we will **do this every time we need to open a terminal to use ROS.**



```
Source /opt/ros/humble/setup.bash
```

- Fortunately, we can automatically trigger it every time we launch a new shell by running the commands.

```
#If using a bash shell
```

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc  
source ~/.bashrc
```

```
#If using a zsh shell
```

```
echo "source /opt/ros/humble/setup.zsh" >> ~/.zshrc  
source ~/.zshrc
```

ROS2 Installation and Configuration (4 / 4)

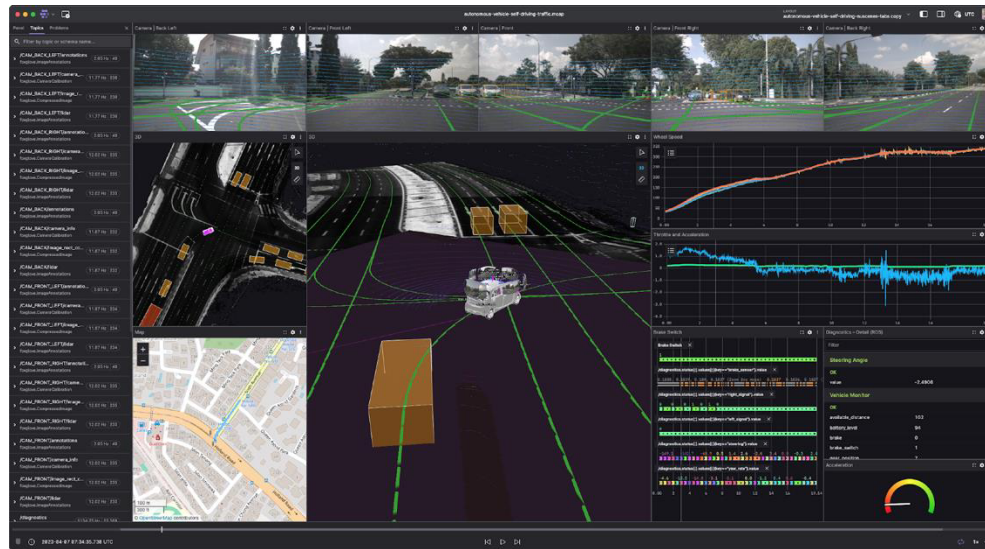
- It is very easy to install any ROS2 packages as they all start with : **ros-humble-**
- You will easily see when you miss some packages, but to begin here is some important one that you might use:

```
#Building applications
sudo apt-get install python3-pip
pip3 install colcon-common-extensions
sudo apt install python3-colcon-common-extensions
#Simulation
sudo apt-get install ros-humble-gazebo-dev
sudo apt-get install ros-humble-gazebo-plugins
sudo apt-get install ros-humble-gazebo-ros
#Robot description
sudo apt-get install ros-humble-joint-state-publisher
sudo apt-get install ros-humble-xacro
sudo apt-get install libpcl-dev libyaml-cpp-dev
#Visualize data
sudo apt-get install ros-humble-foxglove-bridge
sudo apt-get install ros-humble-rosbag2
sudo apt install ros-humble-rosbag2-storage-mcap
```

Development tools and software setup (1 / 2)

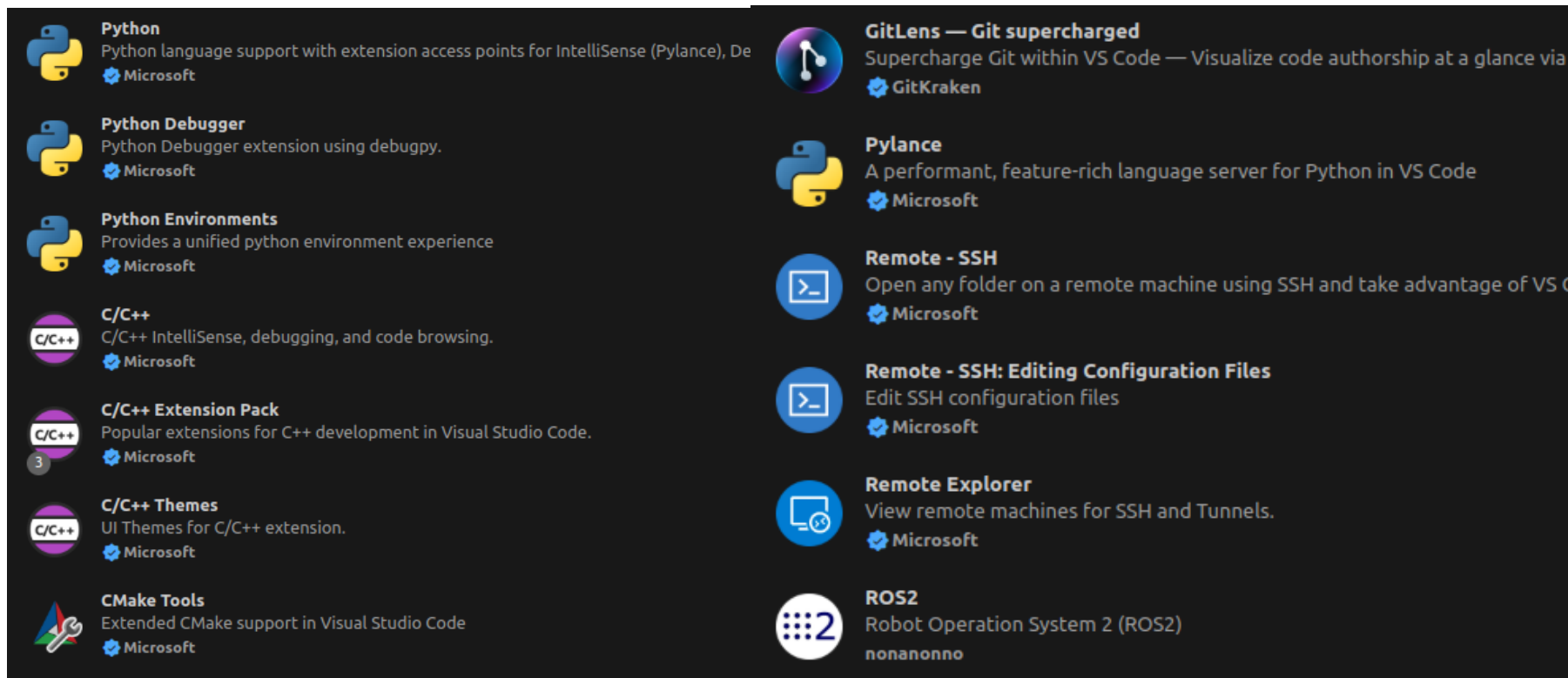
- In this tutorial we will use **foxglove** to visualize data, to easily see what is going on and debug faster, to install the application, click on the link and download the debian package : <https://foxglove.dev/download>
- Then go on the folder where it was downloaded and run:

```
Sudo apt install /foxglove-studio-*.deb
```



Development tools and software setup (2 / 2)

- Finally, when working with ROS2 normally you will develop using VSCode, here is a list of useful extensions to have:



4 ROS2 Basics

Running Executables from existing packages (1 / 3)

- When you want to run a program in ROS, you **run an executable that is located in a packages**. To explain everything about ROS2 basics in the coming section, we will play with a famous packages named **turtlesim**. Moreover if you want to see the packages run the command on a terminal:

```
ros2 pkg list #List of packages
ros2 pkg executables #List of packages and their executables
ros2 pkg executables turtlesim #Executables that we will use from turtlesim
```

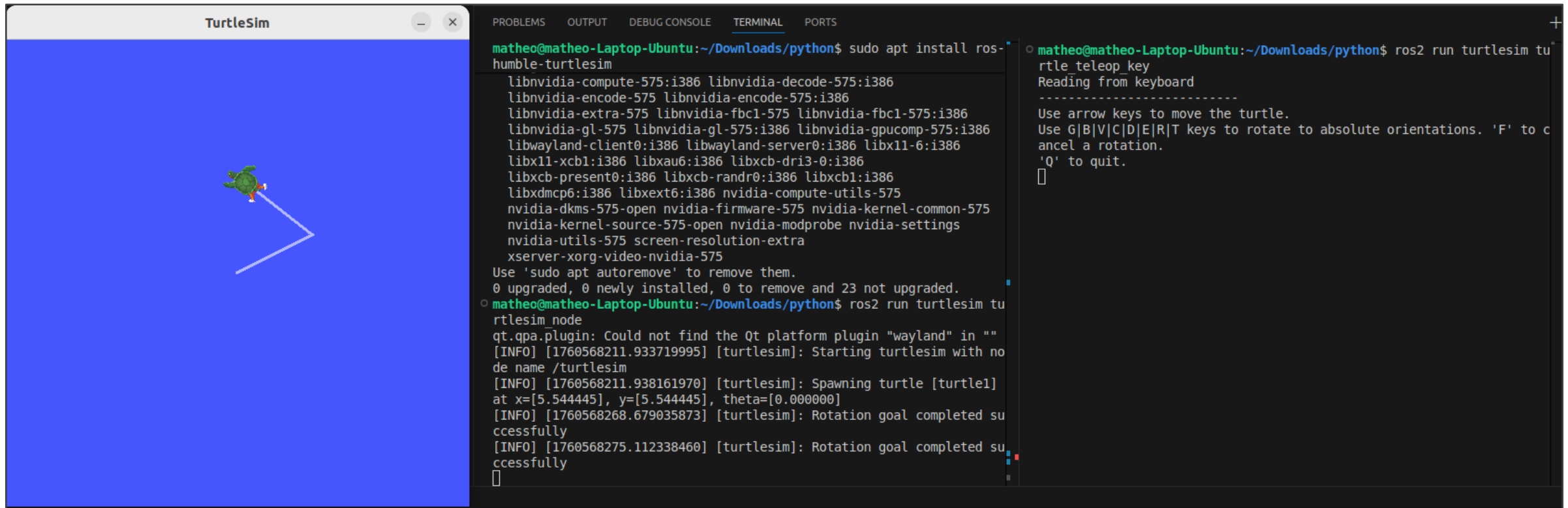
- To run the turtlesim, we can launch by doing the command:

```
sudo apt install ros-humble-turtlesim
ros2 run turtlesim turtlesim_node #Generally : ros2 run <package_name> <executable_name>
```

- To control and interact with the turtle, on another terminal :

```
ros2 run turtlesim turtle_teleop_key
```

Running Executables from existing packages (2 / 3)



The image shows a desktop environment with a window titled "TurtleSim" and a terminal window. The TurtleSim window displays a blue background with a small green turtle icon and a white line indicating its path. The terminal window shows the command `sudo apt install ros-humble-turtlesim` being executed, followed by a list of installed packages. The terminal also shows the command `ros2 run turtlesim turtle_teleop_key` being executed, which starts the turtlesim node and spawns a turtle. The terminal output includes the following text:

```
mattheo@mattheo-Laptop-Ubuntu:~/Downloads/python$ sudo apt install ros-humble-turtlesim
libnvidia-compute-575:i386 libnvidia-decode-575:i386
libnvidia-encode-575 libnvidia-encode-575:i386
libnvidia-extra-575 libnvidia-fbc1-575 libnvidia-fbc1-575:i386
libnvidia-gl-575 libnvidia-gl-575:i386 libnvidia-gpucomp-575:i386
libwayland-client0:i386 libwayland-server0:i386 libx11-6:i386
libx11-xcb1:i386 libxau6:i386 libxcb-dri3-0:i386
libxcb-present0:i386 libxcb-randr0:i386 libxcb1:i386
libxdmcp6:i386 libxext6:i386 nvidia-compute-utils-575
nvidia-dkms-575-open nvidia-firmware-575 nvidia-kernel-common-575
nvidia-kernel-source-575-open nvidia-modprobe nvidia-settings
nvidia-utils-575 screen-resolution-extra
xserver-xorg-video-nvidia-575
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 23 not upgraded.
mattheo@mattheo-Laptop-Ubuntu:~/Downloads/python$ ros2 run turtlesim turtle_teleop_key
Reading from keyboard
-----
Use arrow keys to move the turtle.
Use G|B|V|C|D|E|R|T keys to rotate to absolute orientations. 'F' to cancel a rotation.
'Q' to quit.
[]
```

The terminal window also shows the output of the `ros2 run turtlesim` command, which includes the following text:

```
mattheo@mattheo-Laptop-Ubuntu:~/Downloads/python$ ros2 run turtlesim
qt.qpa.plugin: Could not find the Qt platform plugin "wayland" in ""
[INFO] [1760568211.933719995] [turtlesim]: Starting turtlesim with no de name /turtlesim
[INFO] [1760568211.938161970] [turtlesim]: Spawning turtle [turtle1] at x=[5.544445], y=[5.544445], theta=[0.000000]
[INFO] [1760568268.679035873] [turtlesim]: Rotation goal completed successfully
[INFO] [1760568275.112338460] [turtlesim]: Rotation goal completed successfully
[]
```

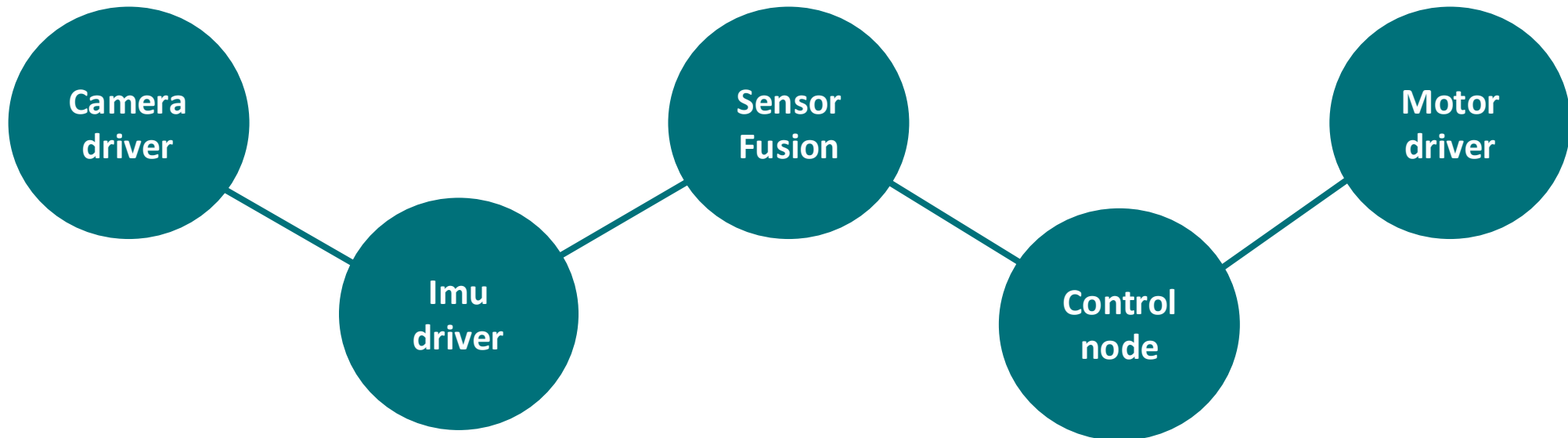
Running Executables from existing packages (3 / 3)

- Another way of running executables is with recorded data that is published (in database).
- The idea is to play back data from the past, maybe replay some sensors data, and change program by testing the modification on the data recorded.
- This record are called **Ros Bag** and to use them:

```
#Record the data
ros2 bag record <topic_name>
#Details about the recording
ros2 bag info <bag_file_name>
#Playback recorded data
ros2 bag play <bag_file_name>
```


Nodes (1 / 4)

- A ROS system is made of **building blocks**, they can be **algorithms**, connectors or **sensors**. Each of these blocks is called a **node**. It can use the ROS libraries and is designed to **perform a specific task** as part of the larger system.
- Nodes are **individual processes that do not share memory**.
- Examples of nodes for a system like a drone:



Nodes (2 / 4)

- We will now see the CLI to have information about nodes, keeping the turtlesim example and launching as before :

```
#List of the current node running
ros2 node list
#Examine a specific node and its different interactions
ros2 node info /turtlesim #Generally ros2 node info <nodeName>
```

```
mattheo@mattheo-Laptop-Ubuntu:~/Downloads/python$ ros2 run turtlesim turtlesim_node
qt.qpa.plugin: Could not find the Qt platform plugin "wayland" in ""
[INFO] [1760569833.557739969] [turtlesim]: Starting turtlesim with node name /turtlesim
[INFO] [1760569833.559907872] [turtlesim]: Spawning turtle [turtle1] at x=[5.544445], y=[5.544445], theta=[0.000000]
[]

mattheo@mattheo-Laptop-Ubuntu:~/Downloads/python$ ros2 node list
/turtlesim
mattheo@mattheo-Laptop-Ubuntu:~/Downloads/python$ ros2 node info /turtlesim
/turtlesim
Subscribers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /turtle1/cmd_vel: geometry_msgs/msg/Twist
Publishers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /rosout: rcl_interfaces/msg/Log
  /turtle1/color_sensor: turtlesim/msg/Color
  /turtle1/pose: turtlesim/msg/Pose
Service Servers:
  /clear: std_srvs/srv/Empty
  /kill: turtlesim/srv/Kill
  /reset: std_srvs/srv/Empty
  /spawn: turtlesim/srv/Spawn
  /turtle1/set_pen: turtlesim/srv/SetPen
  /turtle1/teleport_absolute: turtlesim/srv/TeleportAbsolute
  /turtle1/teleport_relative: turtlesim/srv/TeleportRelative
  /turtlesim/describe_parameters: rcl_interfaces/srv/DescribeParameters
  /turtlesim/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
```

Nodes (3 / 4)

- A node like a function is defined with its parameters. **ROS2 Parameters** are the configuration value of a node.
- They can be considered as **options to change the behaviour of a node**. The programmer who wrote the node can specify whatever parameters they want the users to be able to set.
- To have information about parameters or even set a parameter:

```
#see list of parameters
ros2 param list
#Get parameter value
ros2 param get <node_name> <parameter_name>
#Set parameter value
ros2 param set <node_name> <parameter_name> <value>
```

Nodes (3 / 4)

- A node like a function is defined with its parameters. **ROS2 Parameters** are the configuration value of a node.
- They can be considered as **options to change the behaviour of a node**. The programmer who wrote the node can specify whatever parameters they want the users to be able to set.
- To have information about parameters or even set a parameter:

```
#see list of parameters
ros2 param list
#Get parameter value
ros2 param get <node_name> <parameter_name>
#View all parameters for a node
ros2 param dump <node_name>
#Set parameter value
ros2 param set <node_name> <parameter_name> <value>
```

Nodes (4 / 4)

- The parameters of a node can also be stored or load from a yaml file (we will see later that they are very use for configuration).

```
#Store in a yaml file
ros2 param dump <node_name> > <file.yaml>
#Load from a yaml
ros2 param load <node_name> <file.yaml>
```

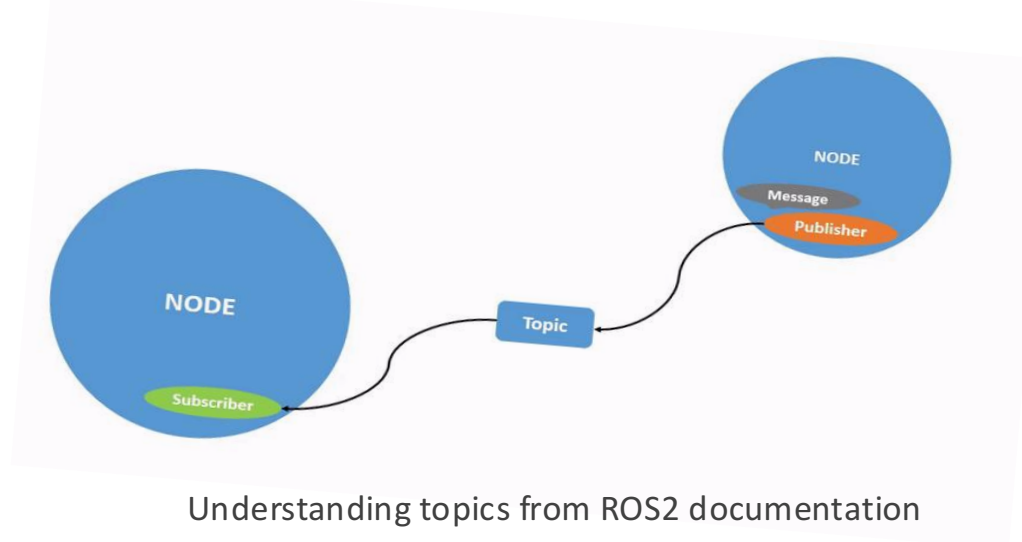
- Finally we can also run an executable with specific parameters loaded on startup.

```
ros2 run <package_name> <executable_name> --ros-args -params-file <file_name>
```

Topics and messages communication (1 / 2)

Only 1 publisher but can have many subscribers

- There are three ways ROS nodes communicate with each other. The most famous one is with **topics** and messages. It allows a message to be routed via a **publisher / subscribers**. With ROS we don't even care if the publisher and the subscriber nodes are even on the same computer (ROS is magic)!
- The **messages are written in the ROS Message Description Language (.msg)**, the common are available on Github.
- In terms of programming all we need to do is specify the name of the topic and the type of message that will be published.



To debug or see more informations :

```
#List of the current topics
ros2 topic list
#To see the type associated
ros2 topic list -t
#View topic freq
ros2 topic hz <topic_name>
```

Topics and messages communication (2 / 2)

- To see more information about an interface:

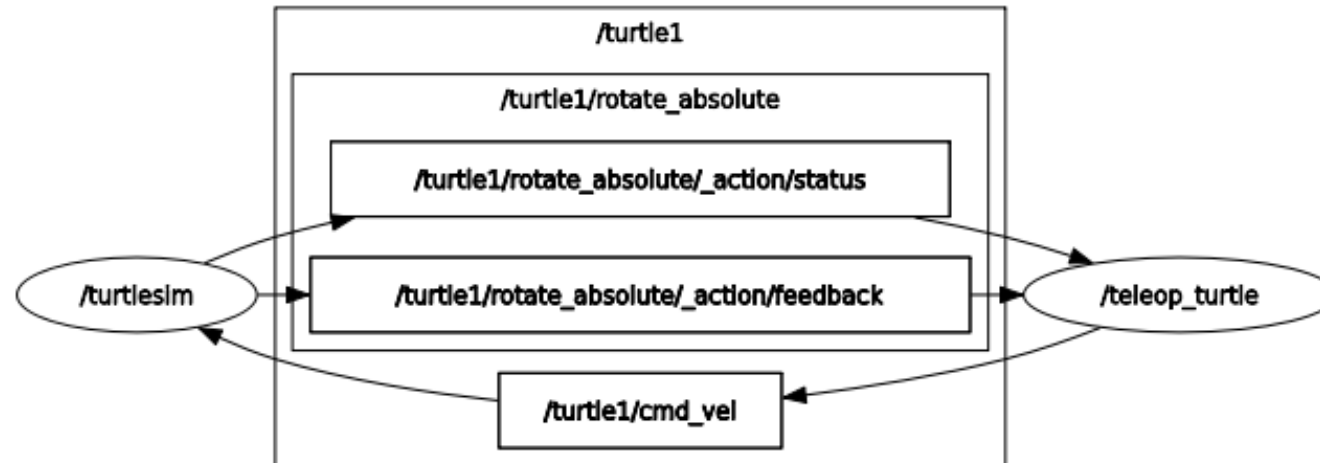
```
#ros2 interface show <type>  
ros2 interface show geometry_msgs/msg/Twist
```

- Publish data to a topic from the command line:

```
#ros2 topic pub <topic_name> <msg_type> '<args>'  
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}" #Publish once and exit  
ros2 topic pub --rate 1 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}" #Publish data with rate 1 hz
```

Computation graph

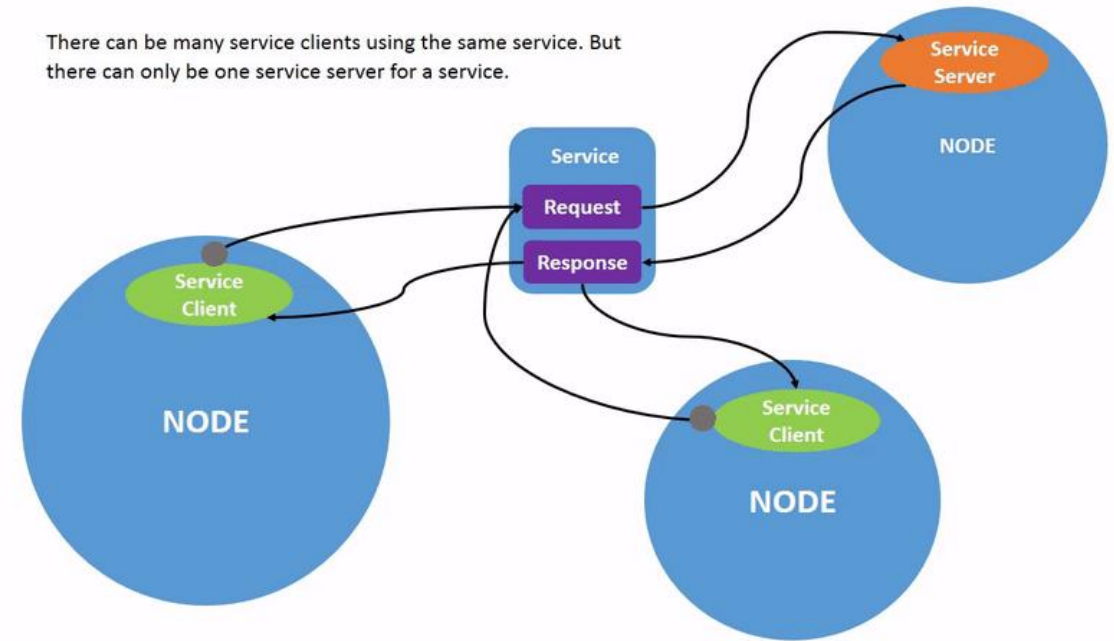
- The computation Graph is the **peer-to-peer network of ROS** processes that **compute data together**. It combines: nodes, messages, topics, bags, services, etc.
- In the graph, nodes are represented with rectangles and communication between nodes are represented by arrows and ellipses.
- We can visualize the graph by using the interface : `rqt_graph`



Others communication (1 / 3)

Services

- Services are another way to communicate but it offers a **single request / reply communication** from one node to another. It is a synchronous communication.
- Moreover, it is **more robust** than topic as it has a response message, so a robust error handling.
- It is commonly used for tasks setting parameters, requesting data, or triggering actions.



Understanding services from ROS2 documentation

Others communication (2 / 3)

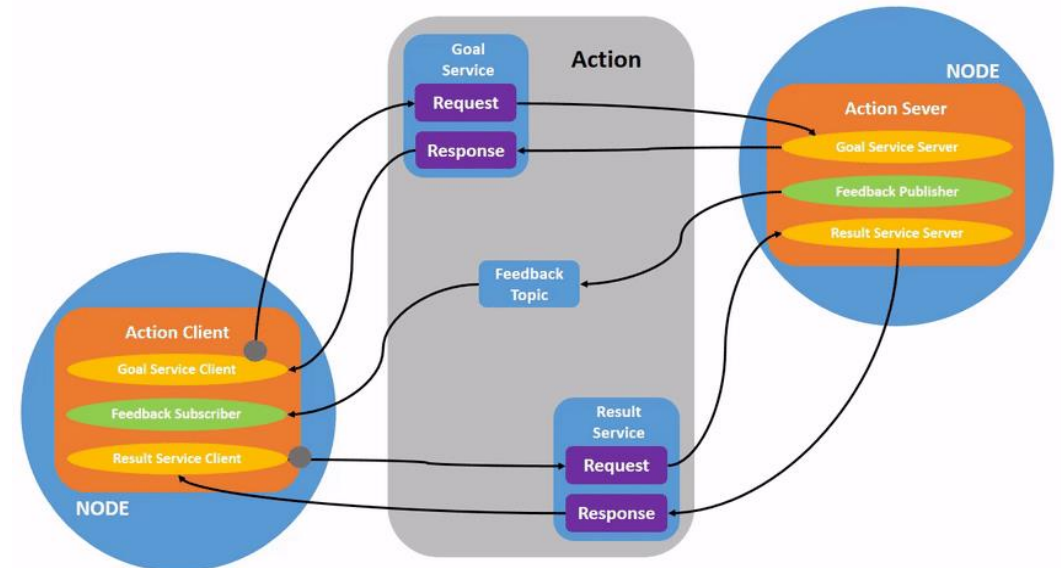
Common services command

```
#List all service names
ros2 service list
#See service type
ros2 service type <service_name>
#Find service with specific type
ros2 service find <service_type>
#see interface
ros2 interface show <service_type>
#Callinf a service
ros2 service call <service_name> <service_type> <arguments>
```

Others communication (3 / 3)

- It is the last way of communicating. They are **build on Topic and services**. It consists of 3 parts:
 - First you set a **goal** via a service (A value to reach).
 - Then the **Feedback** topic gives information about the evolution.
 - Finally the **result** service can inform at any moment about the status.
- Intended for **long running, asynchronous tasks** by providing feedback. For example it can be use for path planning or robot navigation.

```
#List of the current action
ros2 actionlist
#See action info
ros2 action info <action_name>
#See data for action
Ros2 interface show <action_name>
#Send action goal
ros2 action send_goal <action_name>
<action_type> <goal>
```



5 Workspace and Package development

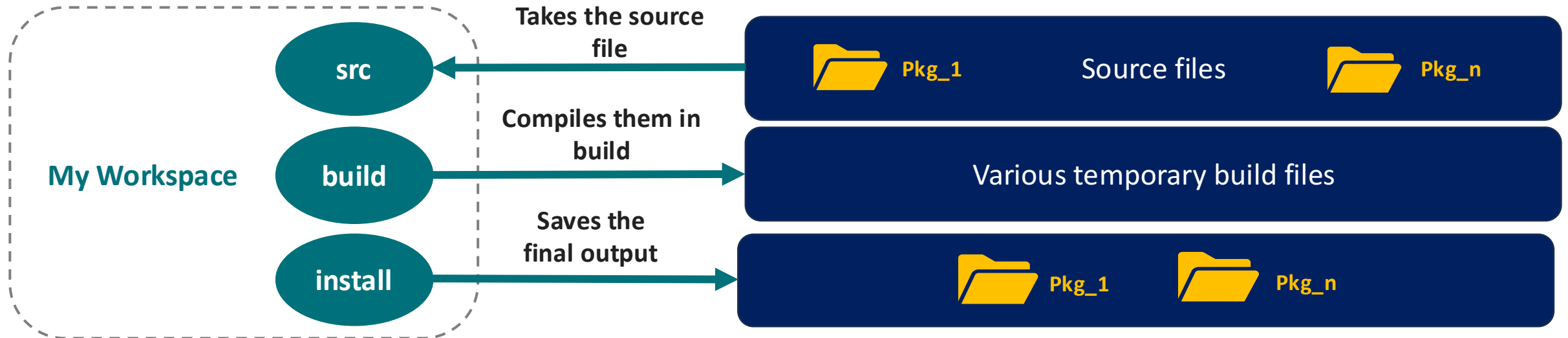
Workspace creation and build systems

(1 / 4)

- Up to now, **we have been users, not developers**. We explored ROS2 basics using pre-built packages like turtlesim, running existing nodes and interacting with topics through command line tools.
- Now we want to become developers, for this reason we will **create our own packages**, write **custom nodes**, **define topics** and understand the complete **development workflow**: workspace structure, build system (colcon), dependencies, compilation and how all components integrate. For this we will create a simple publisher / subscriber in C++.
- When writing your first nodes, you might be tempted to copy-paste code to save time. However, by doing the **effort of understanding simple examples line by line** at the beginning will accelerate your learning curve and enable you to design complex systems (drone, autonomous vehicles, robots, etc).

Workspace creation and build systems (2 / 4)

- When we work on ROS projects, we create something called a **workspace** to **keep all our own packages in** (better to have one workspace per project).
- To build what is inside a workspace we use the build tool **colcon**, because it will know where to **find all the ROS components**. In the first place it will take all our **source files from a src directory**, build them into a **build directory**, and install them into an **install directory**. It also creates a **setup.bash** file (called the **overlay**) with all information to find our packages. So **every time**, like the system installation we will **need to source : path/workspace/setup.bash**



Workspace creation and build systems (3 / 4)

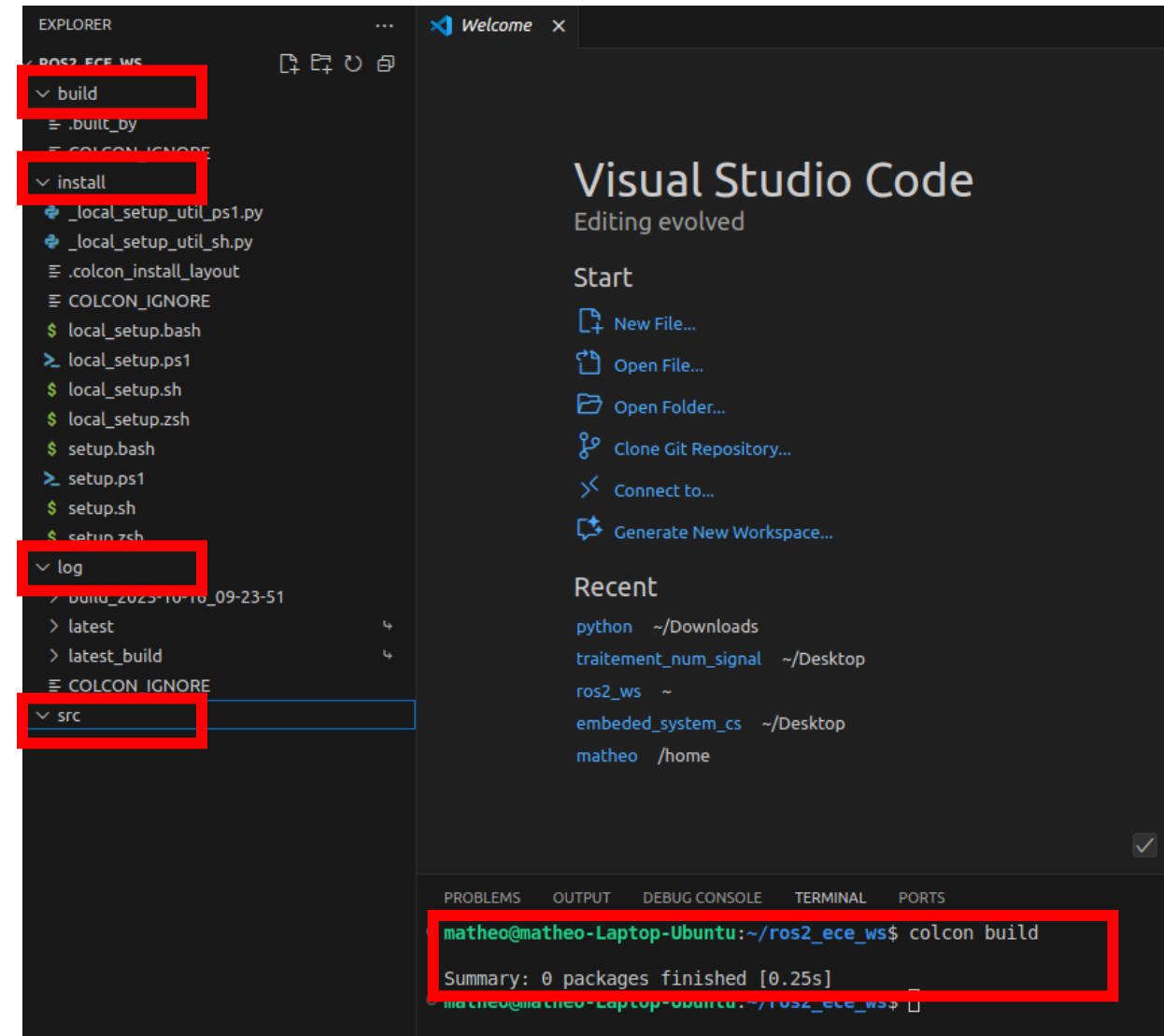
Creation of workspace

```
#1 - If not already done, to use colcon
echo source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash >> ~/.bashrc
#2 - Create the folders
mkdir ros2_ece_ws
cd ros2_ece_ws
mkdir src #Exactly this name
#3 - Always colcon on the workspace directory
#Make sure to be on the workspace folder
colcon build
ros2 service type <service_name>
ls #You can see the folders we talked about earlier (+log containing info about compile)
#4 - Source your overlay
source ~/ros2_ece_ws/install/setup.bash
echo source ~/ros2_ece_ws/install/setup.bash >> ~/.bashrc #Will be done automatically when
opening new term
```

Workspace creation and build systems (4 / 4)

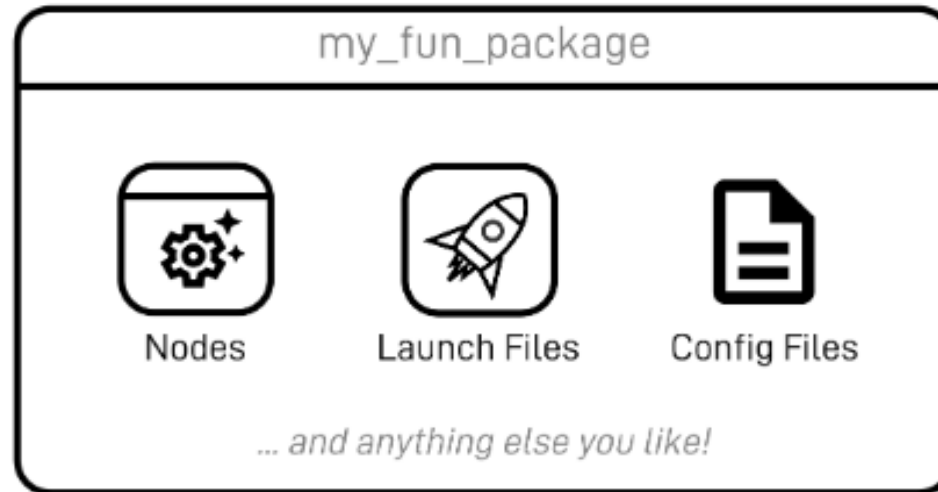
View from VSCode

- Interesting to have VSCode to easily see every folders and files.
- Summary 0 packages as we did not create one yet.
- Ready to create our own custom package!



Package creation and structure (1 / 8)

- A package allow to organize the code and dependencies. It also allows us to reuse common code in different projects.
- It can contain many things such as as one or many nodes, configuration files, launch file, or even 3D models if you want to have your robot description implemented.
- A single workspace can contain many packages. Within any packages we can see many languages of code, for example a launch file can be written either in Python (most common), XML or YAML.



Packages structure from articulated robotics.

Package creation and structure (2 / 8)

Creation of a package

- The first important thing is that we create any package inside the source file src.

```
cd ~/ros2_ece_ws/src
#For a C package
ros2 pkg create --build-type ament_cmake cpp_pubsub
#Full form : ros2 pkg create --build-type ament_cmake --node-name <node> --dependencies <dep>
<pkg_name>
#For a Python package
#ros2 pkg create --build-type ament_python --node-name <node> <pkg_name>
#To build a specific package
cd .. #Go back to the workspace file
colcon build --packages-select cpp_pubsub
```

Package creation and structure (3 / 8)

- Publisher C++ Code make sure to create publisher_member_function.cpp inside ~/ros2_ece_ws/src/cpp_pubsub/src

```
#include <chrono>
#include <functional>
#include <memory>
#include <string>
#include "rclcpp/rclcpp.hpp" //Include most common piece of
ROS2 system
#include "std_msgs/msg/string.hpp" //Include message type to
publish data

using namespace std::chrono_literals;
class MinimalPublisher : public rclcpp::Node //Create the
node
{
public:
MinimalPublisher()
: Node("minimal_publisher"), count_(0)
{
publisher_ = this-
>create_publisher<std_msgs::msg::String>("topic", 10);
timer_ = this->create_wall_timer(
500ms, std::bind(&MinimalPublisher::timer_callback, this));
}
private:
```

```
void timer_callback() //Data set and publish message
{
auto message = std_msgs::msg::String();
message.data = "Hello, world! " + std::to_string(count_++);
RCLCPP_INFO(this->get_logger(), "Publishing: '%s'",
message.data.c_str()); //Print in console
publisher_->publish(message);
}

rclcpp::TimerBase::SharedPtr timer_; //Timer declaration
rclcpp::Publisher<std_msgs::msg::String>::SharedPtr
publisher_;
size_t count_;
};

int main(int argc, char * argv[])
{
rclcpp::init(argc, argv);
rclcpp::spin(std::make_shared<MinimalPublisher>());
rclcpp::shutdown();
return 0;
}
```

Make sure to analyze the code!

Package creation and structure (4 / 8)

- Subscriber C++ Code make sure to create subscriber_member_function.cpp inside ~/ros2_ece_ws/src/cpp_pubsub/src

```
#include <memory>

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"
using std::placeholders::_1;

class MinimalSubscriber : public rclcpp::Node
{
public:
    MinimalSubscriber()
    : Node("minimal_subscriber")
    {
        subscription_ = this->create_subscription<std_msgs::msg::String>(
            "topic", 10, std::bind(&MinimalSubscriber::topic_callback, this,
            _1)); //Create the topic
    }
}
```

```
private:
void topic_callback(const std_msgs::msg::String::SharedPtr msg)
const //Respond to what is given back
{
    RCLCPP_INFO(this->get_logger(), "I heard: '%s'", msg->data.c_str());
}

rclcpp::Subscription<std_msgs::msg::String>::SharedPtr
subscription_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<MinimalSubscriber>());
    rclcpp::shutdown();
    return 0;
}
```

Make sure to analyze the code!

Package creation and structure (5 / 8)

CMakeLists.txt

- Configure **CMakeLists.txt** (setup.py is equivalent) that is located inside src/cpp_pubsub.
- This file **tells colcon and CMake** everything it **needs to build and install our package**. So in our case give the package used and the publisher / Subscriber nodes.
- So in your CMakeLists.txt file make sure to have:

```
if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
|   add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# find dependencies
find_package(ament_cmake REQUIRED)

#Make sure to add this part :-----
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

#publisher
add_executable(talker src/publisher_member_function.cpp)
ament_target_dependencies(talker rclcpp std_msgs)

#Subscriber
add_executable(listener src/subscriber_member_function.cpp)
ament_target_dependencies(listener rclcpp std_msgs)

install(TARGETS
|   talker
|   listener
|   DESTINATION lib/${PROJECT_NAME}
|#-----|-----

if(BUILD_TESTING)
|   find_package(ament_lint_auto REQUIRED)
```

Package creation and structure (6 / 8)

Package.xml

- Before launching we also need to **update other package informations**, using package.xml that will provide extra information.
- The important part in it is the dependencies. These are other packages that we use to build and execute.
- Make sure to add the two dependencies we use in the package.

```
cpp_pubsub > package.xml
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
<package format="3">
  <name>cpp_pubsub</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="matheo.delannoy@gmail.com">matheo</maintainer>
  <license>TODO: License declaration</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <!--ADD DEPENDENCIES HERE-->
  <exec_depend>rclcpp</exec_depend>
  <exec_depend>std_msgs</exec_depend>
  <!--END DEPENDENCIES-->

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

Package creation and structure (7 / 8)

Starting the package

- Before starting anything, we need to make sure to have all dependencies well set.
Make sure to go back on workspace directory.

```
cd ~/ros2_ece_ws
#This will install all requireddependencies
rosdep install -i --from-path src --rosdistro humble -y
#We can now build the package adding --symlink-install command that will allow us when doing
small modification to not colcon all the time
colcon build --symlink-install
#Source or start a new terminal
source ~/.bashrc
```

Package creation and structure (8 / 8)

Starting the package

- We will now start publisher on a terminal: `ros2 run cpp_pubsub talker`
- And subscriber on another: `ros2 run cpp_pubsub listener`

```
mattheo@mattheo-Laptop-Ubuntu:~/ros2_ece_ws$ ros2 run cpp_pubsub talker
[INFO] [1760605364.249066461] [minimal_publisher]: Publishing: 'Hello, world! 0'
[INFO] [1760605364.749113235] [minimal_publisher]: Publishing: 'Hello, world! 1'
[INFO] [1760605365.249135713] [minimal_publisher]: Publishing: 'Hello, world! 2'
[INFO] [1760605365.749711750] [minimal_publisher]: Publishing: 'Hello, world! 3'
[INFO] [1760605366.249212017] [minimal_publisher]: Publishing: 'Hello, world! 4'
[INFO] [1760605366.749911276] [minimal_publisher]: Publishing: 'Hello, world! 5'
[INFO] [1760605367.249124915] [minimal_publisher]: Publishing: 'Hello, world! 6'
[INFO] [1760605367.749114852] [minimal_publisher]: Publishing: 'Hello, world! 7'
[INFO] [1760605368.249483320] [minimal_publisher]: Publishing: 'Hello, world! 8'
[INFO] [1760605368.749587971] [minimal_publisher]: Publishing: 'Hello, world! 9'
[INFO] [1760605369.249148741] [minimal_publisher]: Publishing: 'Hello, world! 10'
[INFO] [1760605369.749154578] [minimal_publisher]: Publishing: 'Hello, world! 11'
[INFO] [1760605370.249155315] [minimal_publisher]: Publishing: 'Hello, world! 12'
```

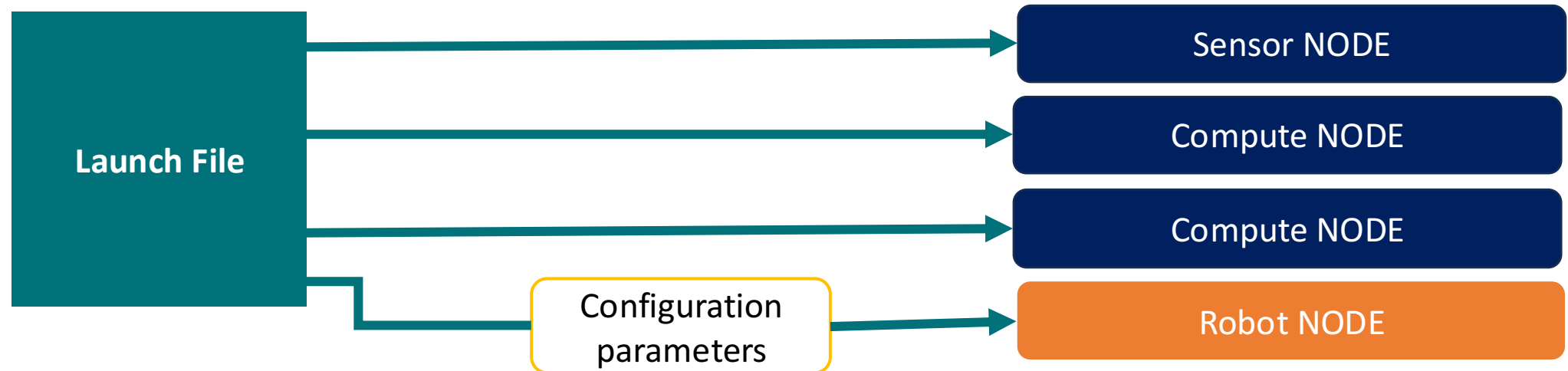
```
mattheo@mattheo-Laptop-Ubuntu:~/ros2_ece_ws$ source ~/.bashrc
mattheo@mattheo-Laptop-Ubuntu:~/ros2_ece_ws$ ros2 run cpp_pubsub listener
[INFO] [1760605365.750116109] [minimal_subscriber]: I heard: 'Hello, world! 3'
[INFO] [1760605366.249860845] [minimal_subscriber]: I heard: 'Hello, world! 4'
[INFO] [1760605366.750390786] [minimal_subscriber]: I heard: 'Hello, world! 5'
[INFO] [1760605367.249478409] [minimal_subscriber]: I heard: 'Hello, world! 6'
[INFO] [1760605367.749490688] [minimal_subscriber]: I heard: 'Hello, world! 7'
[INFO] [1760605368.250038111] [minimal_subscriber]: I heard: 'Hello, world! 8'
[INFO] [1760605368.750008510] [minimal_subscriber]: I heard: 'Hello, world! 9'
[INFO] [1760605369.249593206] [minimal_subscriber]: I heard: 'Hello, world! 10'
[INFO] [1760605369.749585988] [minimal_subscriber]: I heard: 'Hello, world! 11'
[INFO] [1760605370.249518227] [minimal_subscriber]: I heard: 'Hello, world! 12'
```


Launch file (1 / 4)



Launch file format to be detected by colcon :
`<name>_launch.py`

- When you want to launch many nodes at the same time you will need to use `ros2 launch` command.
- ROS2 launch basically executes a launch file that can execute, control and set many things.
- It can be written in XML, YAML or Python. Moreover, the python launch file allows to have better control on synchronization of launch by introducing delay, and there are much easier in terms of structure.
- See an example of what a launch file can do to synchronize the nodes based on configuration parameters.



Launch file (2 / 4)

Launch file program for Publisher / Subscriber

- First, we need to create the launch directory.

```
cd ~/ros2_ece_ws/src/cpp_pubsub  
mkdir launch
```

- Then create a folder pubsub_launch.py with the following code:

```
from launch import LaunchDescription  
from launch_ros.actions import Node  
  
def generate_launch_description():  
    talker = Node(  
        package='cpp_pubsub',  
        executable='talker',  
        name='talker_node',  
        output='screen'  
    )
```

```
    listener = Node(  
        package='cpp_pubsub',  
        executable='listener',  
        name='listener_node',  
        output='screen'  
    )  
  
    return LaunchDescription([talker,  
                               listener])
```

Launch file (3 / 4)

- Before starting make sure that CMakeLists.txt installs the launch files by adding :

```
install(DIRECTORY launch/  
DESTINATION share/${PROJECT_NAME}/launch  
)
```

- Finally, we can build and launch with a different command using launch file : `ros2 launch cpp_pubsub launch.py`

```
• matheo@matheo-Laptop-Ubuntu:~/ros2_ece_ws$ colcon build  
Starting >>> cpp_pubsub  
Finished <<< cpp_pubsub [0.11s]  
  
Summary: 1 package finished [0.28s]  
○ matheo@matheo-Laptop-Ubuntu:~/ros2_ece_ws$ ros2 launch cpp_pubsub pubsub_launch.py  
[INFO] [launch]: All log files can be found below /home/matheo/.ros/log/2025-11-23-17-11-43-742405-matheo-Laptop-Ubuntu-22253  
[INFO] [launch]: Default logging verbosity is set to INFO  
[INFO] [talker-1]: process started with pid [22262]  
[INFO] [listener-2]: process started with pid [22264]  
[talker-1] [INFO] [1763914304.287222697] [talker_node]: Publishing: 'Hello, world! 0'  
[listener-2] [INFO] [1763914304.287794713] [listener_node]: I heard: 'Hello, world! 0'  
[talker-1] [INFO] [1763914304.787553950] [talker_node]: Publishing: 'Hello, world! 1'  
[listener-2] [INFO] [1763914304.787910617] [listener_node]: I heard: 'Hello, world! 1'
```

Launch file (4 / 4)

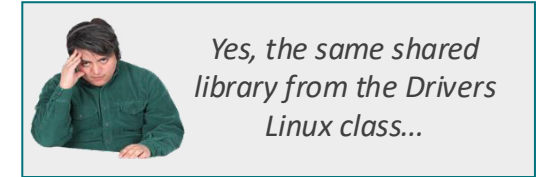
Features of Launch Files

- As you saw earlier, launch files are very important. Instead of launching many nodes, we can launch all of them immediately with the right parameters and a simple command, very interesting for large systems!
- Some interesting features of launch files:

Launch arguments	Running ROSBAG	Execute shell command	Include launch file
If we want to configure behavior at launch time	To automate data logging and show or register the right topic	Running bash command or non-ROS program (start tools)	Avoid large launch file and organize the launch system
<pre> from launch.actions import DeclareLaunchArgument from launch.substitutions import LaunchConfiguration ... rate_arg = DeclareLaunchArgument('rate', default_value='1.0') talker = Node(package='cpp_pubsub', executable='talker', name='talker_node', output='screen', parameters=[{'rate': LaunchConfiguration('rate')}]) </pre>	<pre> ... rosvbag = Node(package='ros2bag', executable='record', arguments=['-o', 'my_bag', '/chatter', '/tf']) return LaunchDescription([talker, listener, rosvbag]) </pre>	<pre> from launch.actions import ExecuteProcess ... #Can execute a bash command log_date = ExecuteProcess(cmd=['bash', '-c', 'echo "Launch started at \$(date)"]', shell=True) #Or even a script (for example to start tools like Gazebo) start_sensor = ExecuteProcess(cmd=['./start_sensor.sh'], cwd='/home/matheo/scripts', output='screen') return LaunchDescription([talker, listener, rosvbag, log_date, start_sensor]) </pre>	<pre> from launch.actions import IncludeLaunchDescription from launch.launch_description_sources import PythonLaunchDescriptionSource from ament_index_python.packages import get_package_share_directory ... IncludeLaunchDescription(PythonLaunchDescriptionSource([get_package_share_directory('some_package'), '/launch/some_launch_file.launch.py'])) </pre>

Plugin creation and usage (1 / 5)

- Plugins are dynamically loadable classes that are loaded from a runtime library without recompiling the node that uses it.
- If you want to understand deeper : **plugins are shared libraries (.so)** registered with pluginlib.
- It is very interesting for robot system as it can:
 - Replace behavior at runtime.
 - Add new features without touching existing code.
 - Support different hardware / sensors easily (interesting for generic algorithms [SLAM]).
 - Choose algorithm implementations using parameters.



What are the different part of plugins?



Plugin creation and usage (2 / 5)

Example of plugin with our Pub/sub package

- First creation of the plugin interface in the include/my_pusub folder →
- Then we implement our different plugins into src directory:

```
#pragma once
#include <string>
namespace cpp_pubsub
{
    class MessageProcessor
    {
    public:
        virtual ~MessageProcessor() = default;
        virtual std::string process(const std::string & input) = 0;
    };
} // namespace cpp_pubsub
```

```
#include "cpp_pubsub/message_processor.hpp"
#include <algorithm>
#include <pluginlib/class_list_macros.hpp>
namespace cpp_pubsub_plugins
{
    class UppercaseProcessor : public cpp_pubsub::MessageProcessor
    {
    public:
        std::string process(const std::string & input) override
        {
            std::string out = input;
            std::transform(out.begin(), out.end(), out.begin(), ::toupper);
            return out;
        }
    };
}
PLUGINLIB_EXPORT_CLASS(cpp_pubsub_plugins::UppercaseProcessor,
cpp_pubsub::MessageProcessor)
```

uppercase_processor.cpp

```
#include "cpp_pubsub/message_processor.hpp"
#include <algorithm>
#include <pluginlib/class_list_macros.hpp>
namespace cpp_pubsub_plugins
{
    class ReverseProcessor : public cpp_pubsub::MessageProcessor
    {
    public:
        std::string process(const std::string & input) override
        {
            return std::string(input.rbegin(), input.rend());
        }
    };
}
PLUGINLIB_EXPORT_CLASS(cpp_pubsub_plugins::ReverseProcessor,
cpp_pubsub::MessageProcessor)
```

Reverse_processor.cpp

Plugin creation and usage (3 / 5)

- Add Plugin XML on package directory: **cpp_pubsub_plugins.xml**

```
<library path="libcpp_pubsub_plugins">
<class name="cpp_pubsub_plugins/UppercaseProcessor"
type="cpp_pubsub_plugins::UppercaseProcessor"
base_class_type="cpp_pubsub::MessageProcessor" />
<class name="cpp_pubsub_plugins/ReverseProcessor"
type="cpp_pubsub_plugins::ReverseProcessor"
base_class_type="cpp_pubsub::MessageProcessor" />
</library>
```

- We also need to add to the **CMakeLists.txt**

```
find_package(pluginlib REQUIRED)
add_library(cpp_pubsub_plugins SHARED
src/uppercase_processor.cpp
src/reverse_processor.cpp)
ament_target_dependencies(cpp_pubsub_plugins pluginlib)
pluginlib_export_plugin_description_file(cpp_pubsub cpp_pubsub_plugins.xml)
add_executable(listener src/subscriber_member_function.cpp)
ament_target_dependencies(listener rclcpp std_msgs pluginlib)
include_directories(include)
install(
  DIRECTORY include/
  DESTINATION include)
install(
  FILES cpp_pubsub_plugins.xml
  DESTINATION share/${PROJECT_NAME})
```

Plugin creation and usage (4 / 5)

- We modify the subscriber to add the plugin feature with a parameter to select plugin.

```
#include <memory>
#include <string>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"
#include "pluginlib/class_loader.hpp"
#include "cpp_pubsub/message_processor.hpp"
using std::placeholders::_1;
class MinimalSubscriber : public rclcpp::Node
{
public:
    MinimalSubscriber()
    : Node("minimal_subscriber")
    {
        // Parameter to select plugin
        this->declare_parameter<std::string>(
            "processor_type",
            "cpp_pubsub_plugins/UppercaseProcessor");
        std::string plugin_name =
            this->get_parameter("processor_type").as_string();
        // Create plugin loader
        loader_ = std::make_shared<pluginlib::ClassLoader<cpp_pubsub::MessageProcessor>>(
            "cpp_pubsub",
            "cpp_pubsub::MessageProcessor"
        );
        try {
            processor_ = loader_->createSharedInstance(plugin_name);
            RCLCPP_INFO(get_logger(), "Loaded plugin: %s", plugin_name.c_str());
        } catch (const std::exception & e) {
            RCLCPP_ERROR(get_logger(), "Failed to load plugin: %s", e.what());
        }
        subscription_ = this->create_subscription<std_msgs::msg::String>(
            "topic", 10,
            std::bind(&MinimalSubscriber::topic_callback, this, _1));
    }
};
```

```
private:
void topic_callback(const
std_msgs::msg::String::SharedPtr msg)
{
    std::string output =
        processor_ ? processor_->process(msg->data) :
        msg->data;
    RCLCPP_INFO(this->get_logger(), "Processed:
    '%s'", output.c_str());
}
rclcpp::Subscription<std_msgs::msg::String>::Sh
aredPtr subscription_;
std::shared_ptr<pluginlib::ClassLoader<cpp_pu
bsub::MessageProcessor>> loader_;
std::shared_ptr<cpp_pubsub::MessageProcesso
r> processor_;
};
int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<MinimalSubscrib
er>());
    rclcpp::shutdown();
    return 0;
}
```


Plugin creation and usage (5 / 5)

- Finally in the launch file if we add the parameter to the listener:

```
listener = Node(  
    package='cpp_pubsub',  
    executable='listener',  
    name='listener_node',  
    parameters=[{"processor_type":  
        "cpp_pubsub_plugins/ReverseProcessor"}],  
    output='screen'  
)
```

```
[listener-2] [WARN] [1763923096.526358515] [pluginlib.ClassLoader]: given plugin name 'li  
plugins' should be 'cpp_pubsub_plugins' for better portability  
[listener-2] [INFO] [1763923096.526621451] [listener_node]: Loaded plugin: cpp_pubsub_plu  
seProcessor  
[talker-1] [INFO] [1763923097.026586210] [talker_node]: Publishing: 'Hello, world! 0'  
[listener-2] [INFO] [1763923097.027176685] [listener_node]: Processed: 'HELLO, WORLD! 0'  
[talker-1] [INFO] [1763923097.526412954] [talker_node]: Publishing: 'Hello, world! 1'  
[listener-2] [INFO] [1763923097.526646824] [listener_node]: Processed: 'HELLO, WORLD! 1'  
[talker-1] [INFO] [1763923098.026935163] [talker_node]: Publishing: 'Hello, world! 2'  
[listener-2] [INFO] [1763923098.027330524] [listener_node]: Processed: 'HELLO, WORLD! 2'
```

- If we put the parameter in comment:

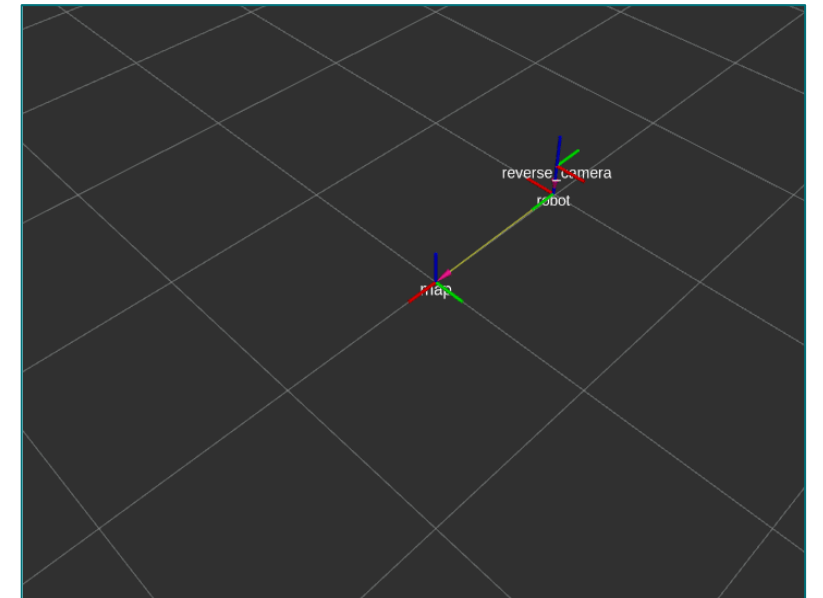
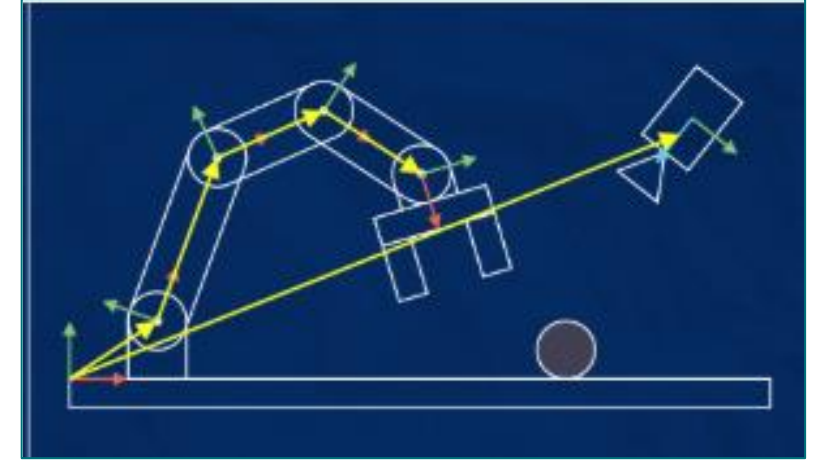
```
listener = Node(  
    package='cpp_pubsub',  
    executable='listener',  
    name='listener_node',  
    #parameters=[{"processor_type":  
        "cpp_pubsub_plugins/ReverseProcessor"}],  
    output='screen'  
)
```

```
[INFO] [talker-1]: process started with pid [53110]  
[INFO] [listener-2]: process started with pid [53120]  
[listener-2] [WARN] [1763929619.670474710] [pluginlib.ClassLoader]: given plugin name 'li  
plugins' should be 'cpp_pubsub_plugins' for better portability  
[listener-2] [INFO] [1763929619.670932955] [listener_node]: Loaded plugin: cpp_pubsub_plu  
Processor  
[talker-1] [INFO] [1763929620.170523214] [talker_node]: Publishing: 'Hello, world! 0'  
[listener-2] [INFO] [1763929620.171215561] [listener_node]: Processed: '0 !dlrow ,olleH'  
[talker-1] [INFO] [1763929620.671234675] [talker_node]: Publishing: 'Hello, world! 1'  
[listener-2] [INFO] [1763929620.671620036] [listener_node]: Processed: '1 !dlrow ,olleH'  
[talker-1] [INFO] [1763929621.170888504] [talker_node]: Publishing: 'Hello, world! 2'  
[listener-2] [INFO] [1763929621.171306615] [listener_node]: Processed: '2 !dlrow ,olleH'
```

6 Robot modeling and simulation

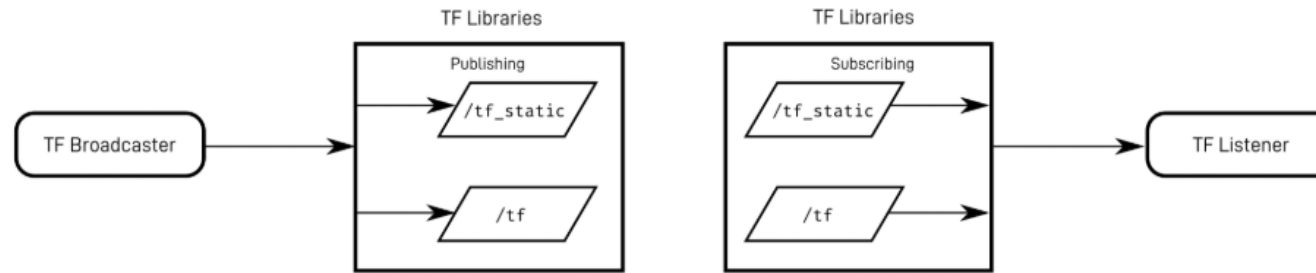
TF2 (1 / 3)

- Coordinate transformations are very important as it allows to **change the representations to avoid very complex and long calculation involving trigonometry.**
- We study an object in a particular frame to easily determine the movement or interaction; we can also have link between multiple coordinate frames which allow us to keep track of multiple coordinate frames over time.
- ROS2 provides a system called **TF2 (TransForm V2)** to handle these transformations for us. Any node can use the library to broadcast a transform from one frame to another.
- Those transforms will form a **tree structure** where each frame is defined by one transform from another frame. Important to **know for every element in a system: position, velocity, others TF etc.**



TF2 (2 / 3)

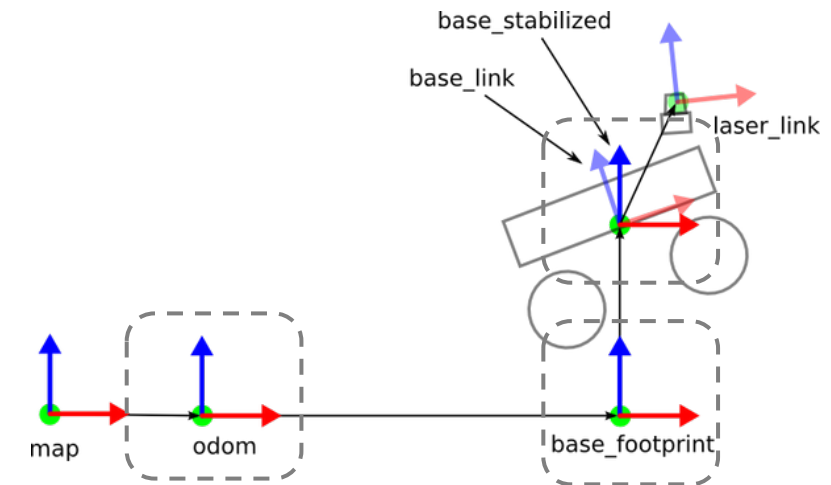
- The TF library are using topics to handle all communication, but as topic are not published by user it is named **broadcasting and listening**.



- **Topics :** **/tf** for dynamic transform like joints and **/tf_static** for static transform like robot geometry.

- Important axes of the tree of TFs:

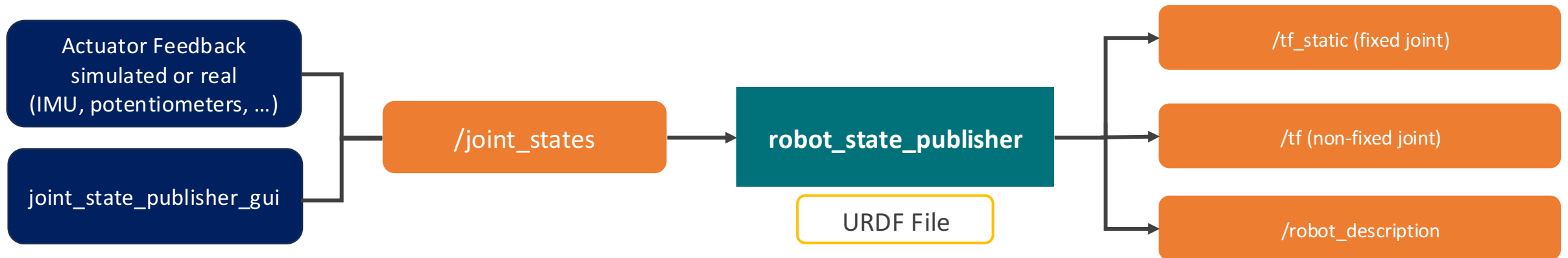
- **/base_footprint** : **root of a robot's TFs** and is the center of the robot on the ground (can be middle of the wheel for a vehicle).
- **/base_link** : Child of base and is typically the **center of a robot**.
- **/odom** : parent of the base and it is the transformation that indicates the **robot's displacement**.



TF2 (3 / 3)

- To see the TF tree while running the code:

```
ros2 run tf2_tools view_frames
```
- TF are usually generated in **URDF files**, which are **organized as a tree** similar to the TF tree. Since they are closely related, there is a ROS node called **robot_state_publisher** that **reads the URDF file and broadcasts the transforms and robot description**.
- But this only handles static transforms. To get **dynamic transforms**, we use **/joint_states** which provide external values like angles and distances from actuator sensors (encoders, IMU, etc). There is also a handy tool called `joint_state_publisher_gui` that shows all movable joints with sliders so you can see changes in real time during development.



URDF (1 / 9)

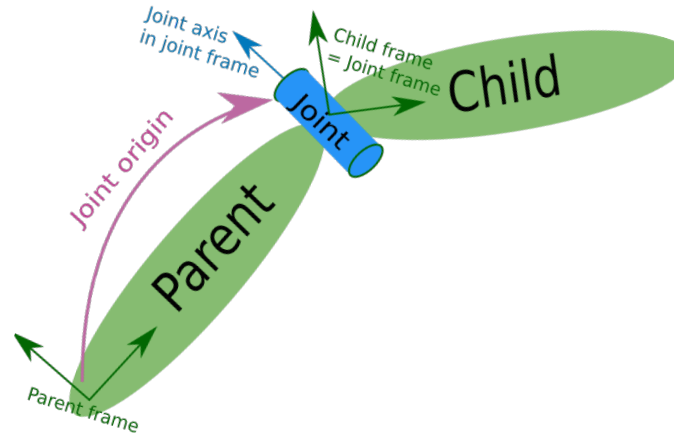
Make sure to use the right nomenclature to name your elements:

- For joints: anyName_**joint**
- For links: anyName_**link**

- URDF stands for **Unified Robot description Format** and is represented in **XML**. As its names can tell it is used to describe the **visual, physics and connections of a robot** with **JOINTS** and **LINKS**.

Joints are used to describe the connections between two links, a parent link and a child link

A **link** is used to describe a segment of a robot where we can define visual, collision and inertial



```
<?xml version="1.0"?>
<robot name="ece_bot">
  <link name="name_link">
    <visual>
    </visual>
    <collision>
    </collision>
    <inertial>
    </inertial>
  </link>

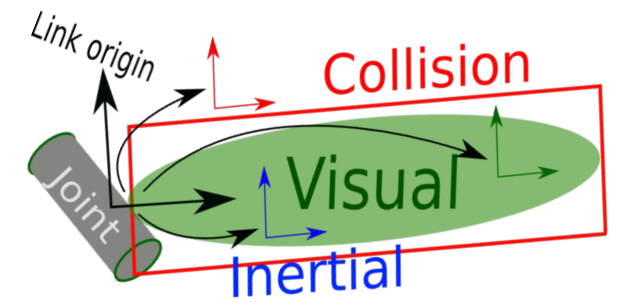
  <joint name="name_joint" ...>
    <axis ... />
    <parent link ... />
    <child link ... />
    <origin ... />
  </joint>
</robot>
```

URDF syntax

URDF (2 / 9)

Links

- Additional characteristics we can specify:
 - **Visual** – For **displaying the robot in RViz or Gazebo**. Define a **geometry** (box, sphere etc. or mesh files like .dae/.stl), **origin** (where it's centered), **material** (color).
 - **Collision** – Used for physics collision calculations. Set **geometry** and **origin** similar to visual but keep it simple (No copy paste). **Use basic shapes, NOT lesches for computing ressources.**
 - **Inertial** – Defines how the link responds to forces. Includes **mass**, **origin (center of mass)** and **inertia (rotational inertia matrix)** for physics calculations.



Some geometries in URDF (SI units)

```
<box size = "x y z"/> <!-- BOX -->
<cylinder radius = "r" length = "l"/> <!-- CYLINDER -->
<sphere radius = "r"/> <!-- SPHERE -->
<mesh filename = "file_path" scale = "x y z"/> <!-- MESH -->
```

Origin rotation applied first and then translation

```
<origin xyz = "x y z" rpy = "r p y"/> <!-- ORIGIN same for visual, collision, origin-->
```

Material define color, transparency etc

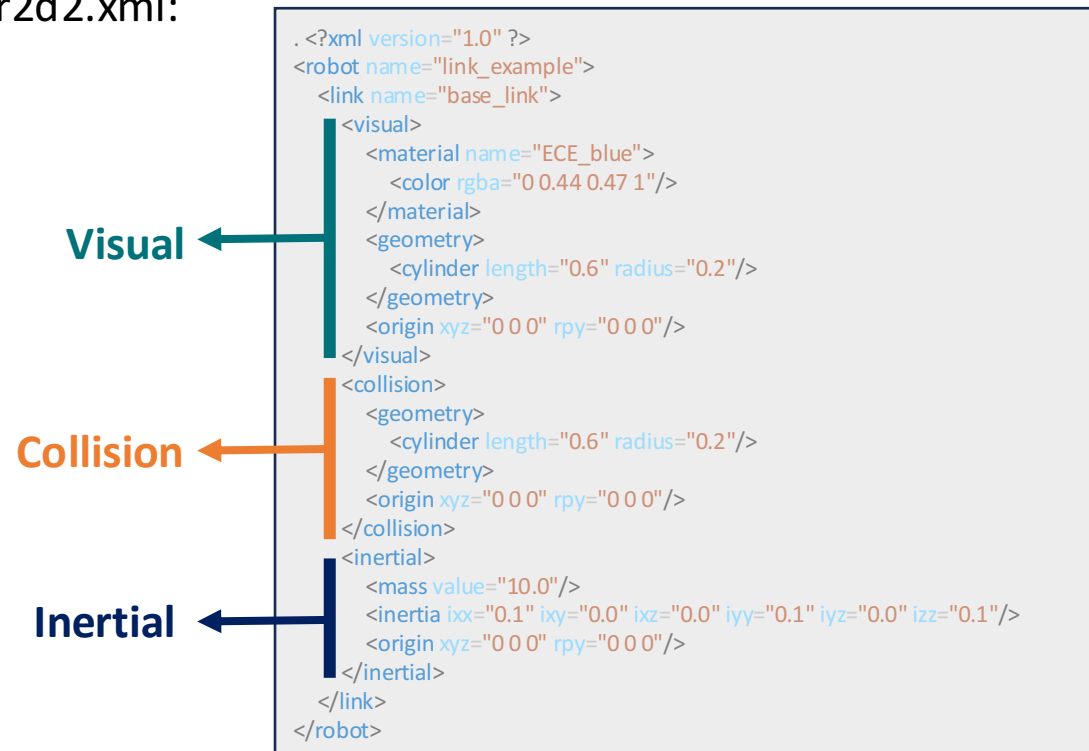
```
<material name = "material_name">
  <color rgba = "r g b a"/> <!-- COLOR -->
  <texture filename = "file_path"/> <!-- TEXTURE -->
</material>
```

Describe inertia in kg.m²

```
<inertial>
  <mass value = "masse_kg"/>
  <inertia ixx = "ixx" ixy = "ixy"
    izx = "izx" izy = "izy" iyy = "iyy"
    iyz = "iyz" izz = "izz"/>
</inertial>
```

URDF (3 / 9)

- The same way as before we create a new package for this part (see part 5 for more information). Make sure to **arrange the files CMakeLists.txt and package.xml** to make the project work!
- We will create a simple link that will be a cylinder, I will name the file r2d2.xml:



The same way as before we create a launch file to easily in one command execute the package

```

from launch import LaunchDescription
from launch_ros.actions import Node
from ament_index_python.packages import get_package_share_directory
import os
  
```

```

def generate_launch_description():
    my_urdf = os.path.join(
        get_package_share_directory('robot_description'),
        'src',
        'r2d2.xml'
    )
  
```

Allow to access xml file

```

with open(my_urdf, 'r') as infp:
    robot_description_content = infp.read()
  
```

Retrieve the content of xml file and make it robot description

```

return LaunchDescription([
    Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        parameters=[['robot_description': robot_description_content]],
    ),
    Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
    ),
])
  
```

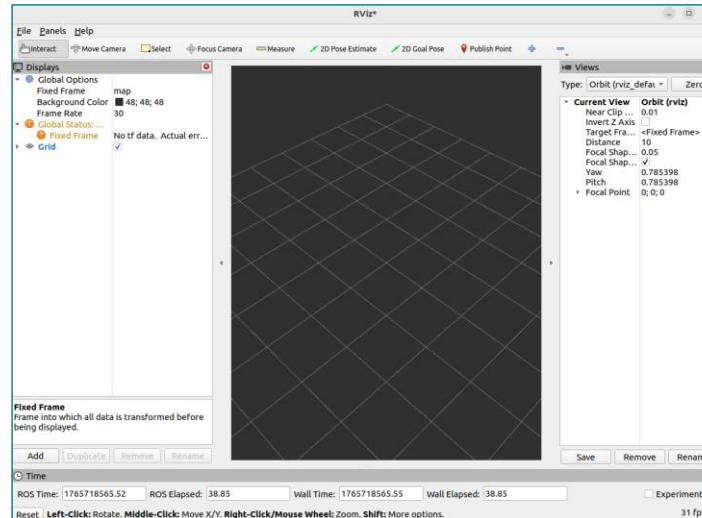
Nodes that will be launch for the robot description and the visualization tool Rviz

URDF (4 / 9)

- Therefore, to execute the project we need again to colcon, source and launch:

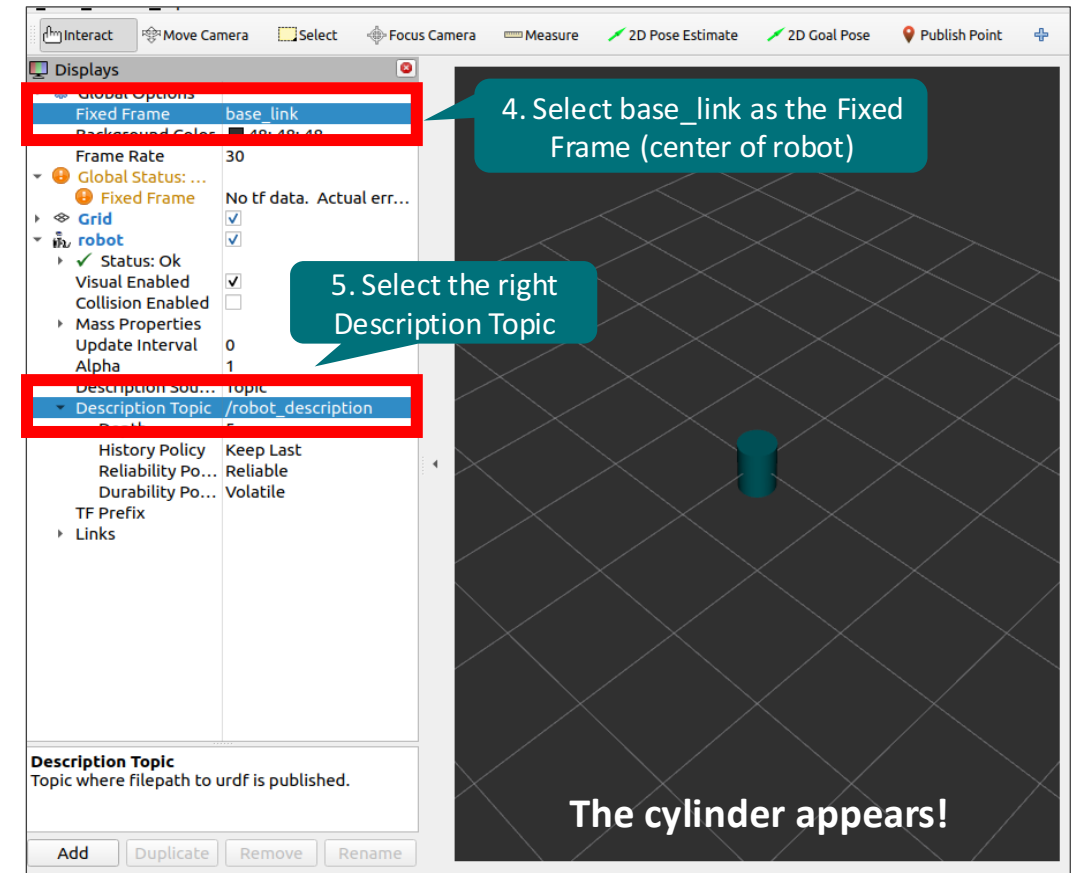
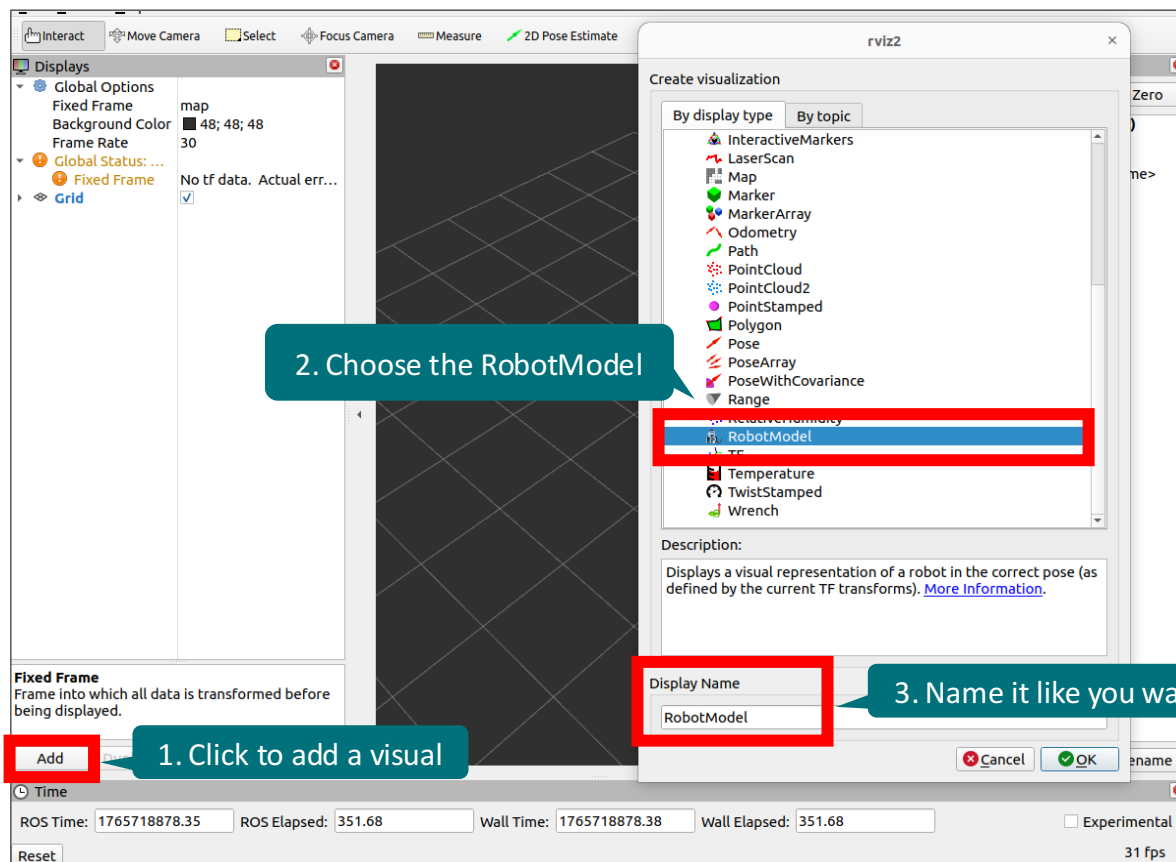
```
colcon build --symlink-install --packages-select robot_description #--symlink install allows to execute without  
# building again (for small changes like modifying robot description  
# --packages-select to only build a specific package (don't need to build the old cpp_pubsub package)  
source install/setup.bash # Can also just go in another terminal as we configured the "bashrc"  
ros2 launch robot_description robot_description_launch.py
```

- When we execute if everything is well installed the RViz window will open!
- However, we don't see the cylinder because the RViz tool is not configure we need to add the robot display.



URDF (5 / 9)

- We will present the RViz visualisation tool in details later but for now we need to do a little bit of configuration to visualize the link.



You can save your configuration and execute it everytime you launch.

URDF (6 / 9)

- First thing to do is to save the configuration in RViz : File > save Config As (Ctrl+Shift+S) and save it in the package for example in the following location: `robot_description->rviz/display.rviz`
- Then we can either open the configuration every time in RViz or directly add it to the launcher file:

```
from launch import LaunchDescription
from launch_ros.actions import Node
from ament_index_python.packages import get_package_share_directory
import os

def generate_launch_description():
    my_urdf = os.path.join(
        get_package_share_directory('robot_description'),
        'src',
        'r2d2.xml'
    )

    rviz_config_file = os.path.join(
        get_package_share_directory('robot_description'),
        'rviz',
        'display.rviz'
    )

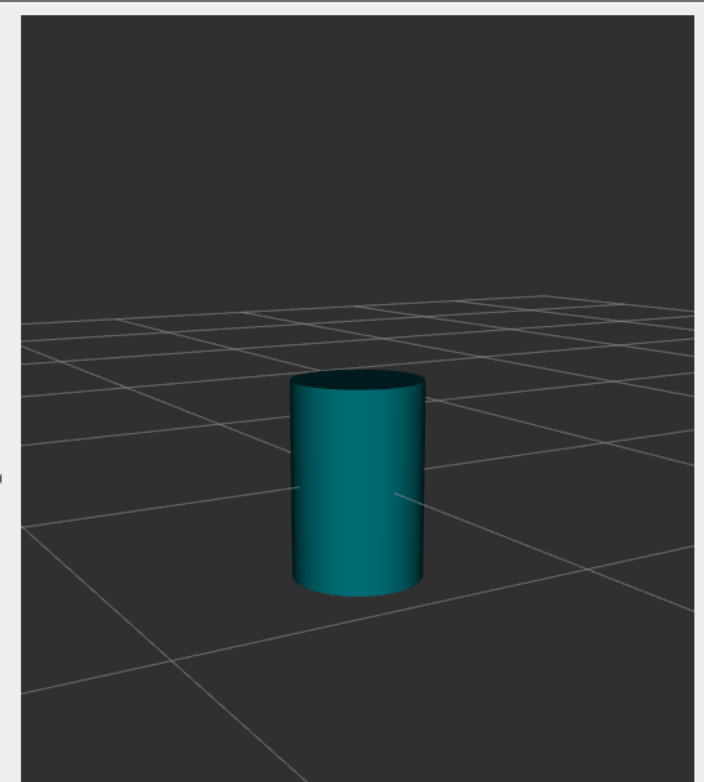
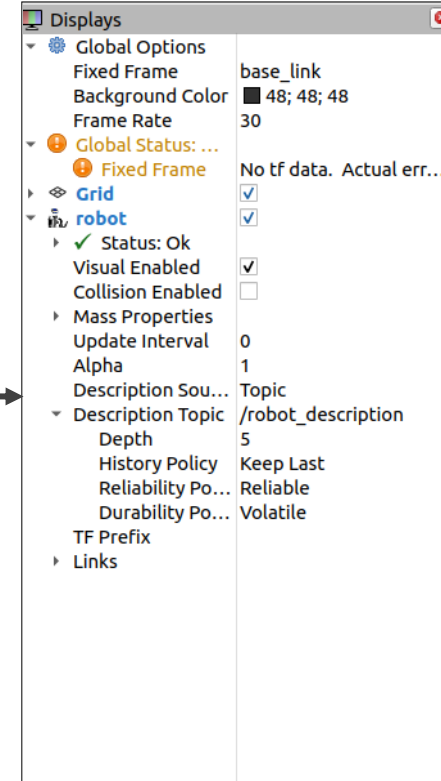
    with open(my_urdf, 'r') as infp:
        robot_description_content = infp.read()
    return LaunchDescription([
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            parameters=[{'robot_description': robot_description_content}],
        ),
        Node(
            package='rviz2',
            executable='rviz2',
            arguments=['-d', rviz_config_file],
            name='rviz2',
        ),
    ])

```

Access the config file

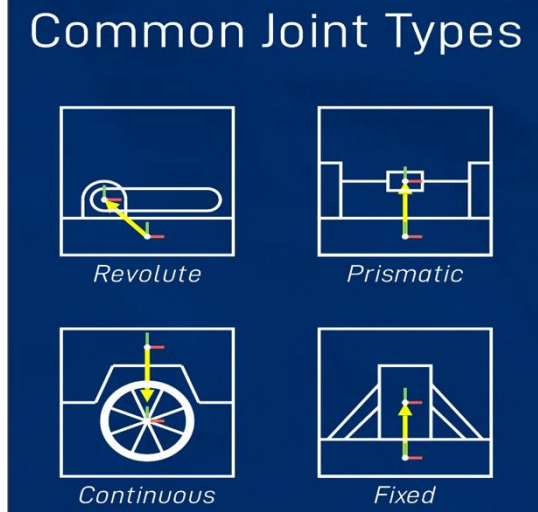
Add the configuration to the rviz node

Launch



Joints

- The joints is the **transforms** to define where the frames are, for that we specify:
 - **Type of Joints** – Different ways to connect two links:
 - **Fixed**: No motion between links
 - **Revolute**: Rotation with limits (radians)
 - **Continuous**: Rotation without limits (ex: wheels)
 - **Prismatic**: Linear motion
 - **Floating**: Interesting to create our own motion (3 translation, 3 rotation)
 - **Parent and Child Link** – Link before and after the joint
 - **Origin and Axis** – Describes the **location of the child frame relative to the parent frame**.
 - **Limit** – For **revolute (rad)** and **prismatic (m)** joints only. Can have an **effort in Nm** and **velocity (maximum in rad/s)**.
 - **Dynamics** – Include **damping (N s/m)** and **friction (N)**.



Common Joint Types from articulated robotics.

URDF (8 / 9)

1. Just modify the `r2d2.xml` file to add new features. First, we install the `joint_state_publisher_gui` tool introduced earlier.
2. Then we add the joint and a child link to the xml file:

```
sudo apt install ros-humble-joint-state-publisher-gui
#We could also add the dependency in package.xml
```

```
<?xml version="1.0" ?>
<robot name="joint_example">
  <material name="ECE_blue">
    <color rgba="00.44 0.47 1"/>
  </material>
  <material name="white">
    <color rgba="1 1 1 1"/>
  </material>
  <link name="base_link">
    <visual>
      <material name="ECE_blue"/>
      <geometry>
        <cylinder length="0.6" radius="0.2"/>
      </geometry>
    </visual>
  </link>
  <link name="right_leg_link">
    <visual>
      <material name="white"/>
      <geometry>
        <box size="0.6 0.1 0.2"/>
      </geometry>
      <origin rpy="0 1.57075 0" xyz="0 0 -0.3"/>
    </visual>
  </link>
  <joint name="base_to_right_leg_joint" type="continuous">
    <parent link="base_link"/>
    <child link="right_leg_link"/>
    <origin xyz="0 -0.22 0.25"/>
    <axis rpy="0 0 0" xyz="0 1 0"/>
  </joint>
</robot>
```

New link

Joint continuous between
base_link and right_leg

3. We also add the `joint_state_publisher_gui` node in the launch file, so the launch description becomes:

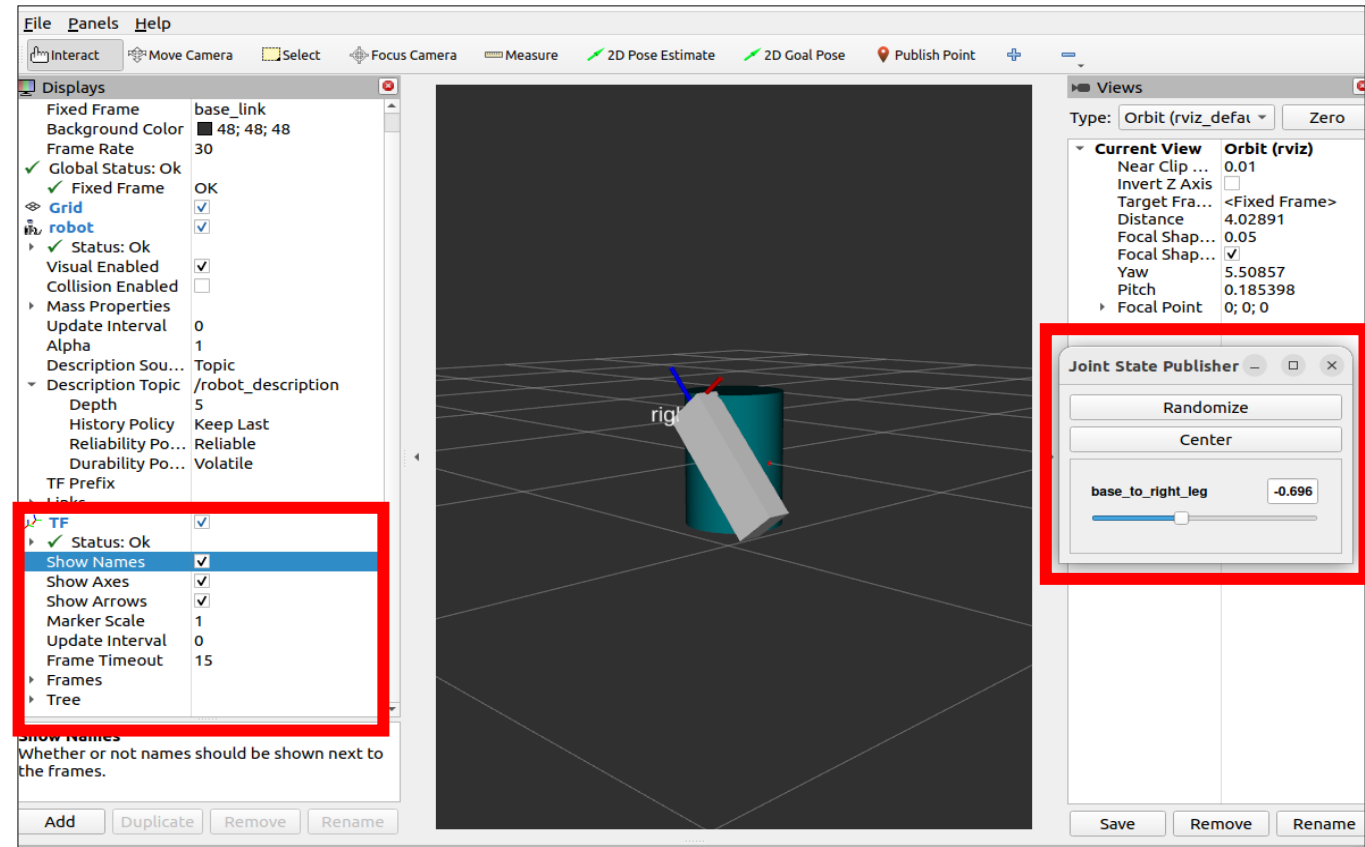
```
return LaunchDescription([
  Node(
    package='joint_state_publisher_gui',
    executable='joint_state_publisher_gui',
    name='joint_state_publisher_gui',
  ),
  Node(
    package='robot_state_publisher',
    executable='robot_state_publisher',
    parameters=[('robot_description': robot_description_content)],
  ),
  Node(
    package='rviz2',
    executable='rviz2',
    arguments=['-d', rviz_config_file],
    name='rviz2',
  ),
])
```

URDF (9 / 9)

- In the end we can build and launch to see the new visual in RViz.
- In the screen also appears a window to adjust all the transforms (joints).
- Finally, if we want to visualize the transforms we must change the configuration: **Add > By Display Type > TF**

As you can observe URDF is simple and explicit, but imagine for a large robot description the size of the XML file...

Xacro



Xacro (1 / 4)

- **Xacro means (XML Macros)** it can be defined as a **preprocessor for URDF**. As it will **generates standard URDF** before being used by ROS.
- It simplifies URDF in many ways, allows to **separate in multiple files** and add a lot of new features such as:
 - **Properties** – Same property shows up at multiple places. Define only once `<xacro:property name="leg_length" value="0.6"/>`
 - **Macros** – Reusable component with parameters (several lines are repeated) `<xacro:macro name="leg" params="name x y">`
 - **Loops** `<xacro:for each="i" in="0 1 2 3">`
 - **Conditionals** `<xacro:if value="$(arg use_lidar)">`
 - **Include** – Other files added to the main `<xacro:include filename="a_file.xacro" />`
- It was created to have great readability for large description and to avoid repetition (**respect DRY**)

Teachers: your code should follow the principle of DRY: Don't Repeat Yourself
My code:



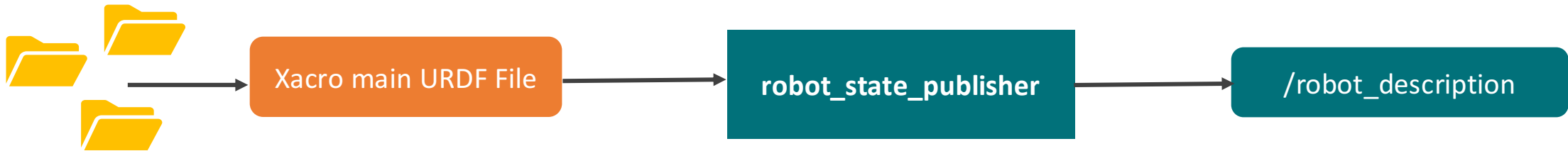
Xacro (2 / 4)

- To **enable the use of Xacro** in a file, we just need to **add a tag** to the declaration of robot at the beginning:

```
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">
```

Put in every robot tag so we can use xacro whenever we want

- Typically, the main **URDF/xacro has the extension .urdf.xacro**, and the **included can have .xacro** only.



Many included, unprocessed
xacro Files (.xacro)

Single processed URDF File
(.urdf.xacro)

Same as classic URDF

- Xacro command to convert a .xacro file to a .urdf file (must be specify in launch file):

```
xacro model.xacro > model.urdf
```

URDF describes robots
Xacro helps humans write URDF efficiently

Xacro (3 / 4)

- To show the different usage of xacro files, we will create an example quite similar as the previous one but with many files.

materials.xacro

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">
<material name="white">
<color rgba="1 1 1 1"/>
</material>
<material name="ECE_blue">
<color rgba="0 0.44 0.47 1"/>
</material>
</robot>
```

body.xacro

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">
<link name="base_link">
<visual>
<geometry>
<cylinder length="0.6" radius="0.25"/>
</geometry>
<material name="ECE_blue"/>
</visual>
</link>
</robot>
```

leg.xacro

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">
<!-- Demonstrate macro to create multiple legs -->
<xacro:macro name="leg" params="name x y z">
<link name="${name}">
<visual>
<geometry>
<box size="0.4 0.1 0.2"/>
</geometry>
<material name="white"/>
</visual>
</link>
<joint name="base_to_${name}" type="continuous">
<parent link="base_link"/>
<child link="${name}"/>
<origin xyz="{x} {y} {z}" rpy="0 0 0"/>
<axis xyz="0 1 0"/>
</joint>
</xacro:macro>
</robot>
```

Macro definition

face.xacro

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">
<link name="head_link">
<visual>
<geometry>
<sphere radius="0.2"/>
</geometry>
<material name="white"/>
</visual>
</link>
<joint name="base_to_head" type="fixed">
<parent link="base_link"/>
<child link="head_link"/>
<origin xyz="0 0 0.5" rpy="0 0 0"/>
</joint>
</robot>
```

r2d2.urdf.xacro (main)

```
<?xml version="1.0"?>
<robot name="mini_r2d2"
xmlns:xacro="http://www.ros.org/wiki/xacro">
<!-- Global properties -->
<xacro:property name="leg_z" value="-0.3"/>
<!-- Includes -->
<xacro:include filename="materials.xacro"/>
<xacro:include filename="body.xacro"/>
<xacro:include filename="face.xacro"/>
<xacro:include filename="leg.xacro"/>
<!-- Legs usage of defined xacro -->
<xacro:leg name="leg_front_left" x="0.2" y="0.2" z="${leg_z}"/>
<xacro:leg name="leg_front_right" x="0.2" y="-0.2" z="${leg_z}"/>
<xacro:leg name="leg_back_left" x="-0.2" y="0.2" z="${leg_z}"/>
<xacro:leg name="leg_back_right" x="-0.2" y="-0.2" z="${leg_z}"/>
</robot>
```

Global property

Includes

Macro usage

I use the same package, but I delete the r2d2.xml file. In the end no repetition, and well-organized robot description.

Xacro (4 / 4)

- Before running the code, we must modify a little bit the launch file to work with the new files and add the xacro command as mentioned earlier.

```
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.substitutions import Command
from ament_index_python.packages import get_package_share_directory
import os

def generate_launch_description():
    xacro_file = os.path.join(
        get_package_share_directory('robot_description'),
        'src',
        'r2d2.urdf.xacro'
    )

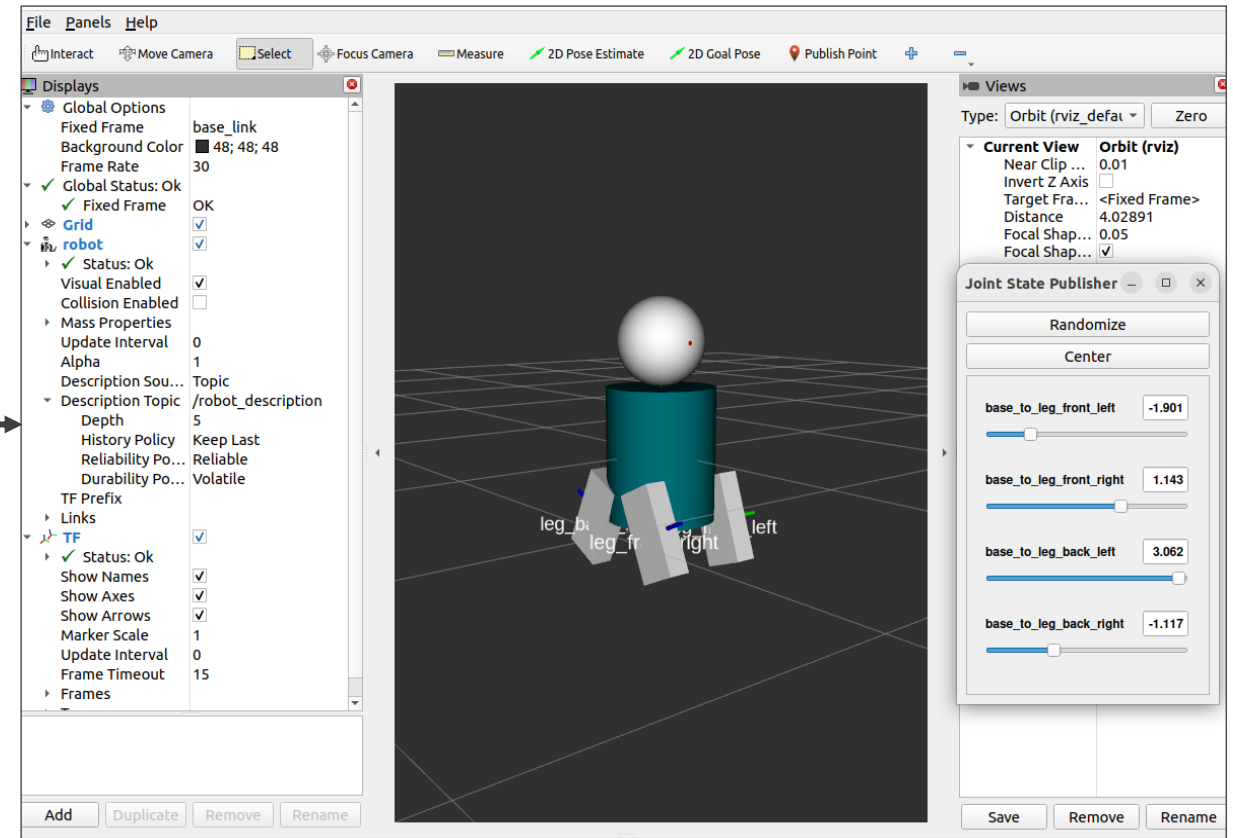
    rviz_config_file = os.path.join(
        get_package_share_directory('robot_description'),
        'rviz',
        'display.rviz'
    )

    #Define the command to process the xacro file
    robot_description = Command(['xacro ', xacro_file])
    return LaunchDescription([
        Node(
            package='joint_state_publisher_gui',
            executable='joint_state_publisher_gui'
        ),
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            parameters=[('robot_description', robot_description)],
        ),
        Node(
            package='rviz2',
            executable='rviz2',
            arguments=['-d', rviz_config_file],
        ),
    ])
```

Xacro command to convert

Usage of xacro command in robot_state_publisher

Build
Launch



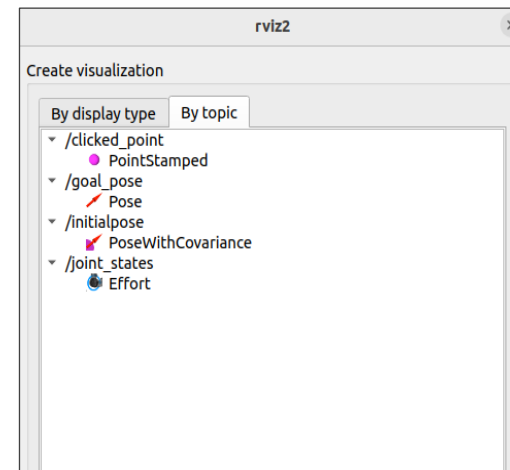
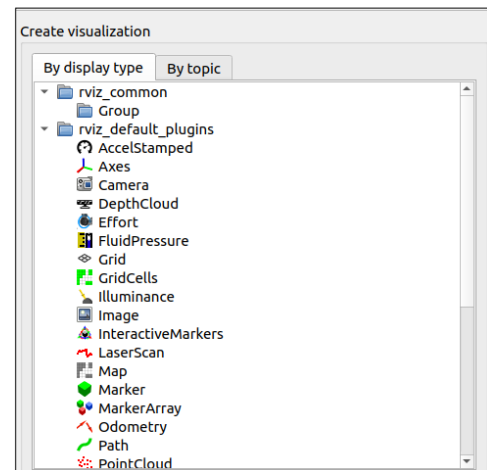
« R2D2 »

Visualization tools (1 / 7)

- We use it earlier in the tutorial and we already know how useful RViz visualizing tool can be to see robot model.
- In addition, RViz can be used to:
 - **Display sensor data** (LaserScan, PointCloud, Camera images) ;
 - Debug **localization, mapping, navigation** with many tools specific to autonomous vehicles ;
 - Play bag recorded data to visualize them (ROS bag) ;
 - Even publish data on specific topic or set a goal with the tool bar.

Can **visualize by display or by topic** (recommended to use by topics because it shows the topics that are available only).

RViz visualizes data through Displays.
Displays subscribe to Topics.



RViz visualizes ROS data, it does not
simulate the robot.

Visualization tools (2 / 7)

- Normally the Foxglove tool was already installed previously, we will learn now how to use it.
- Foxglove is a modern, web-based **visualization and debugging tool** for robotics data. We can inspect, visualize, replay and debug ROS2 projects in real time or offline with recorded data. It is very powerful and easy to handle, **I recommend working with Foxglove instead of RViz to have great visuals.**
- Unlike RViz, **Foxglove is not tied to ROS internals and can work with multiple data sources.**
- It is a **Websocket** (requires to have only 1 port open on ROS, for that it requires an extra node named **foxglove_bridge**).
- It is organized with **Layouts** and **Panels**.

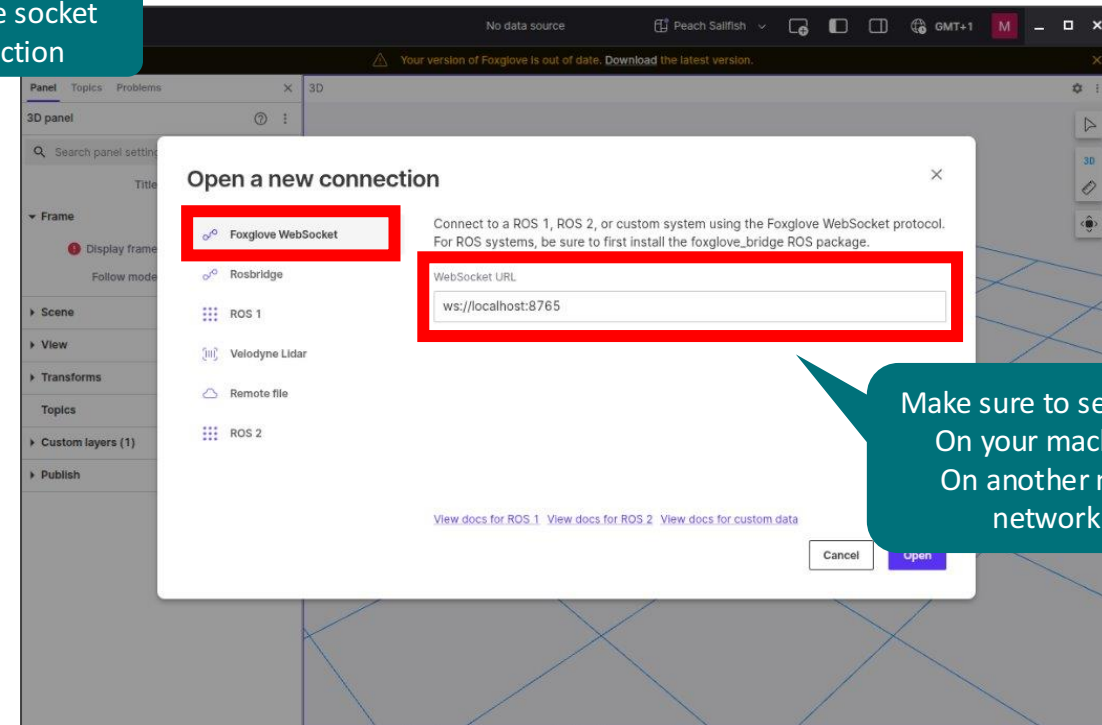
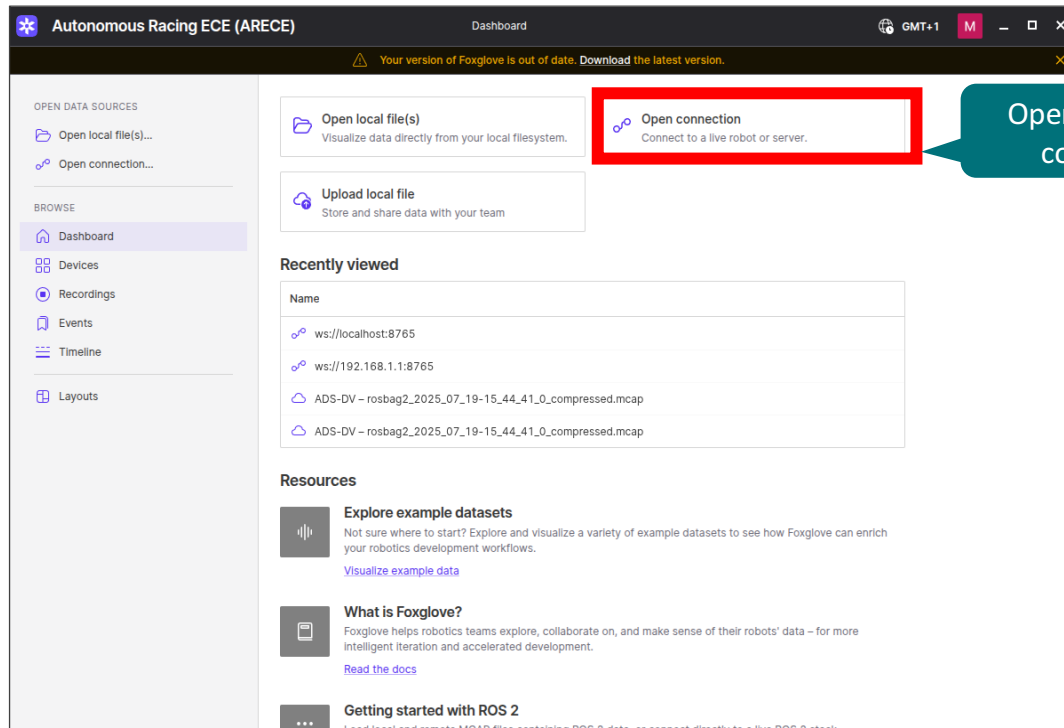
Layouts is the screen with all the panels they can be save to be reuse in order to avoid configuring all the panels at each launch. They can be shared across an organization.

Panels are visualization interfaces with many possibilities such as 3D, Image, Plot, Map, publish, raw messages, etc. List of available in "add panel" menu.

Visualization tools (3 / 7)

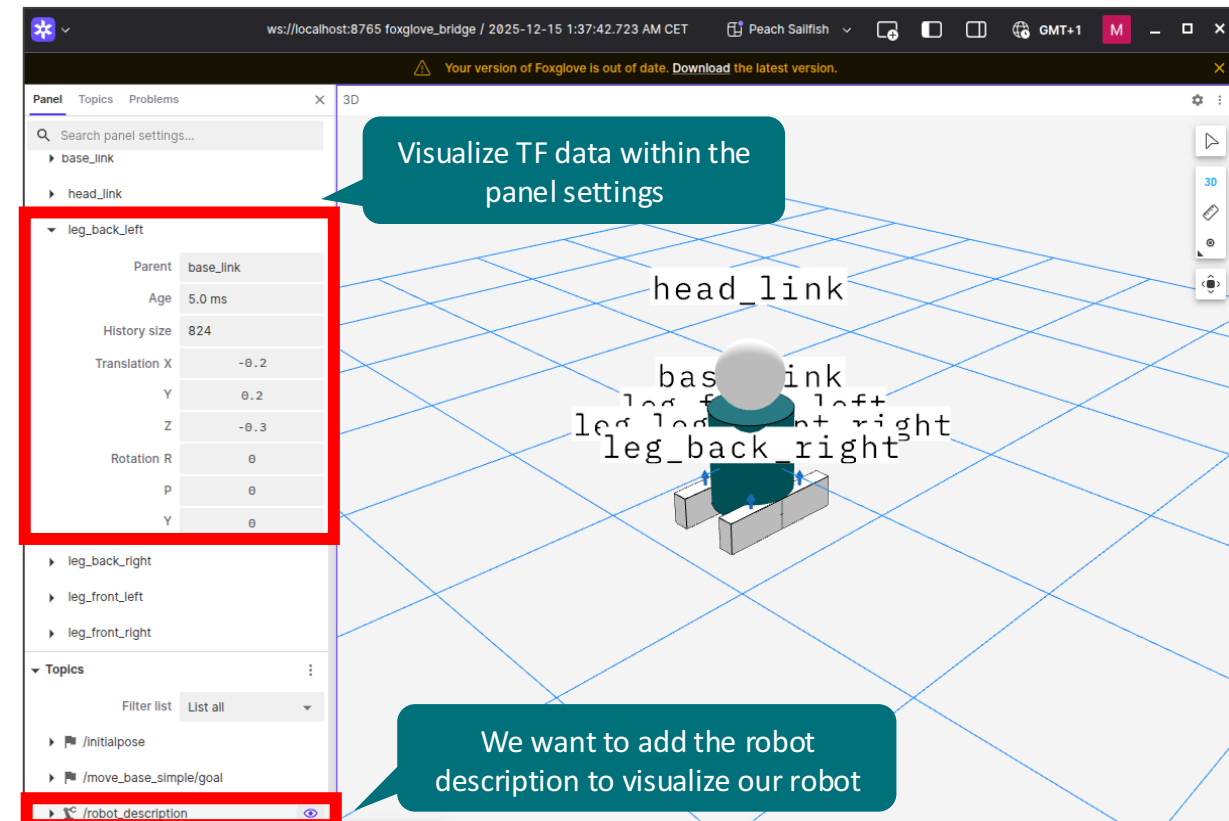
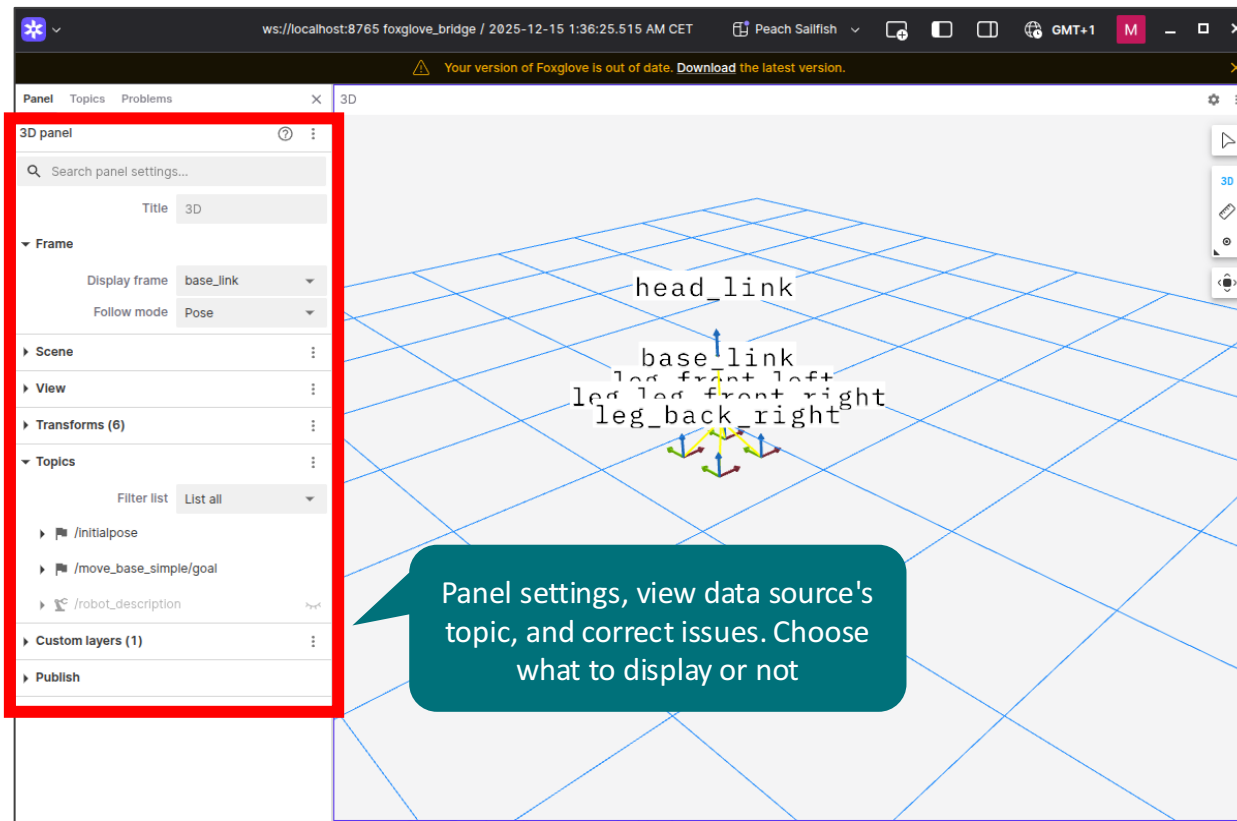
- We will just modify the launch file of our R2D2 project to use **Foxglove** instead of RViz. Replace the **RViz node** with the following:
- After building and launching we need to open the Foxglove application:

```
# Bridges ROS 2 data to Foxglove
Node(
  package='foxglove_bridge',
  executable='foxglove_bridge',
  name='foxglove_bridge',
  output='screen'
),
```



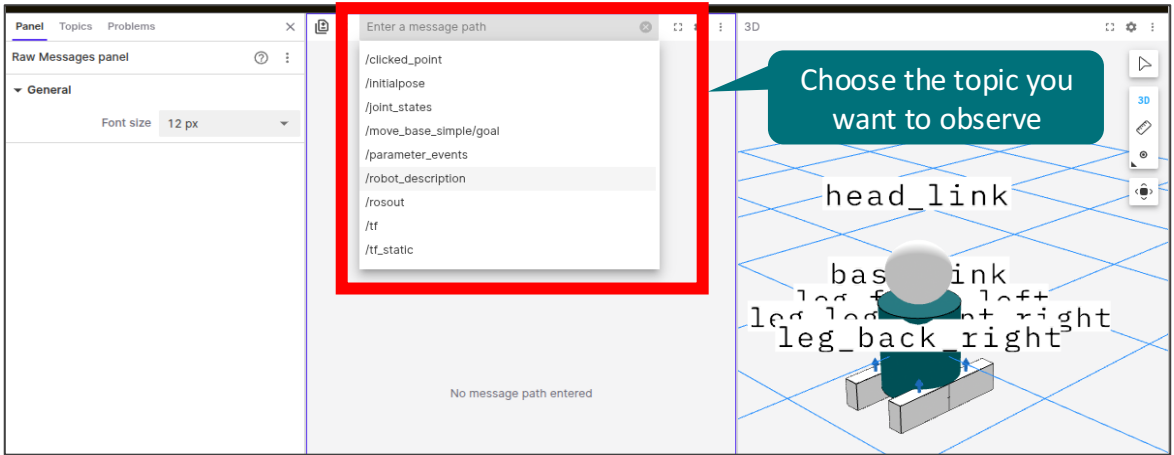
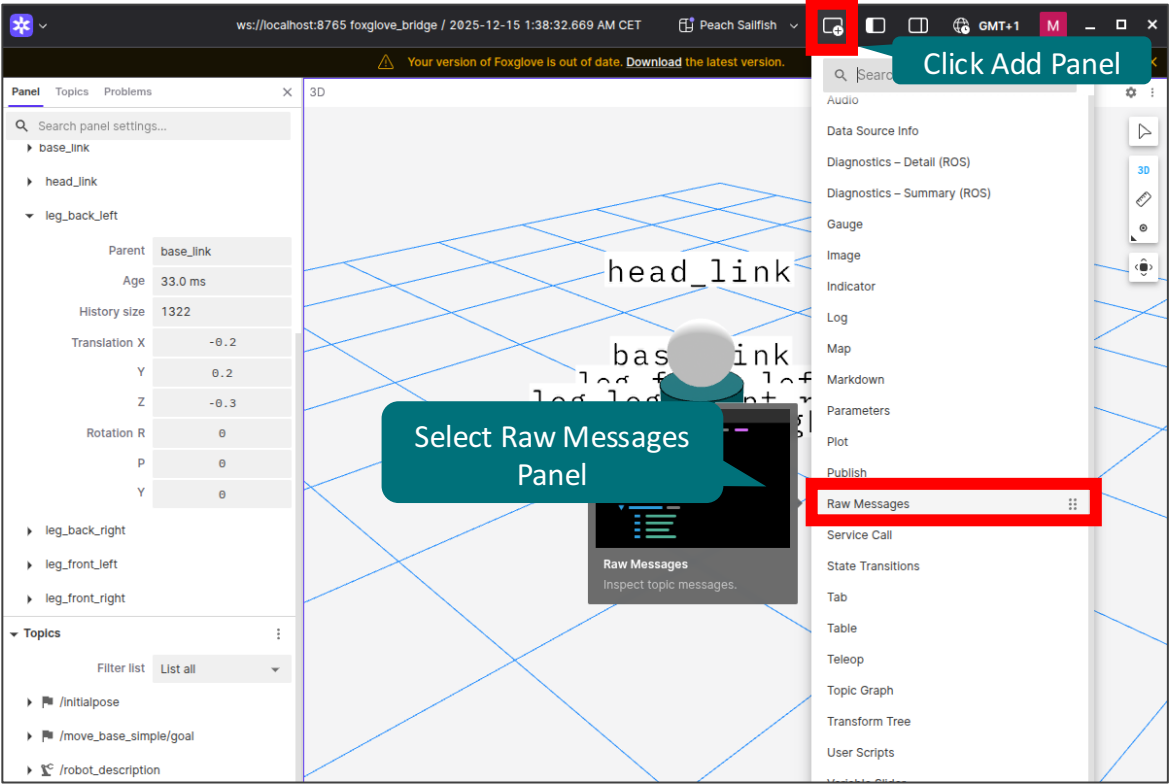
Visualization tools (4 / 7)

- After connected our ROS2 stack to Foxglove we can start visualizing with a 3D panel:



Visualization tools (5 / 7)

- To better understand the power of the tool we will add a new panel for example the raw message to also understand better what our TF2 system publish.



Visualization tools (6 / 7)

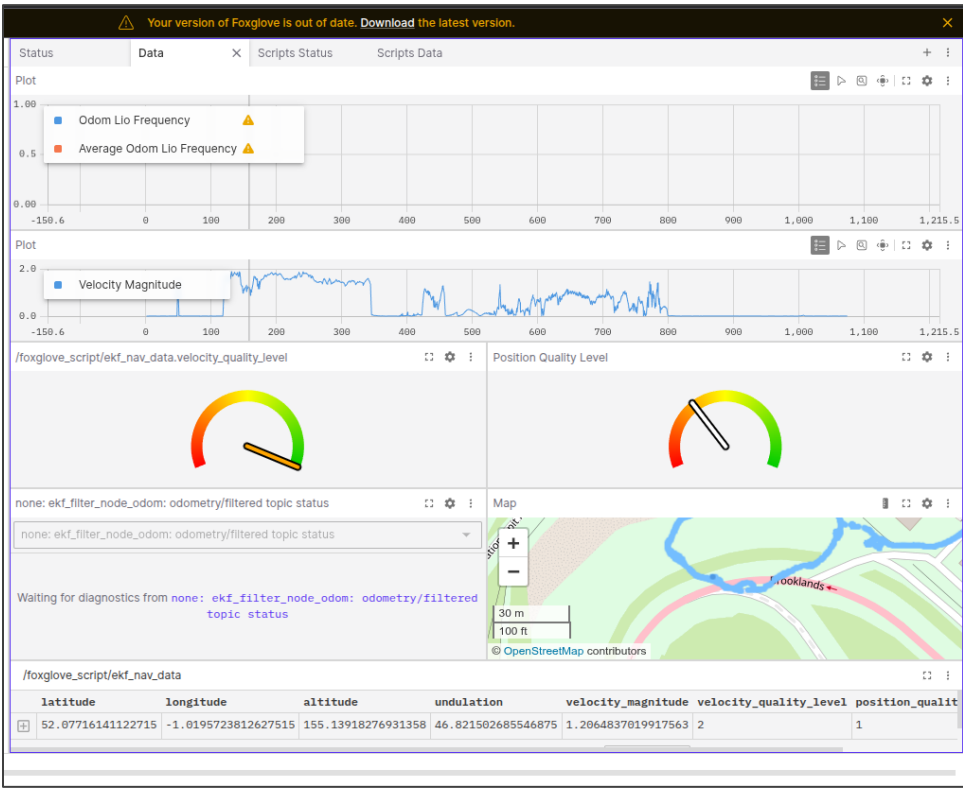
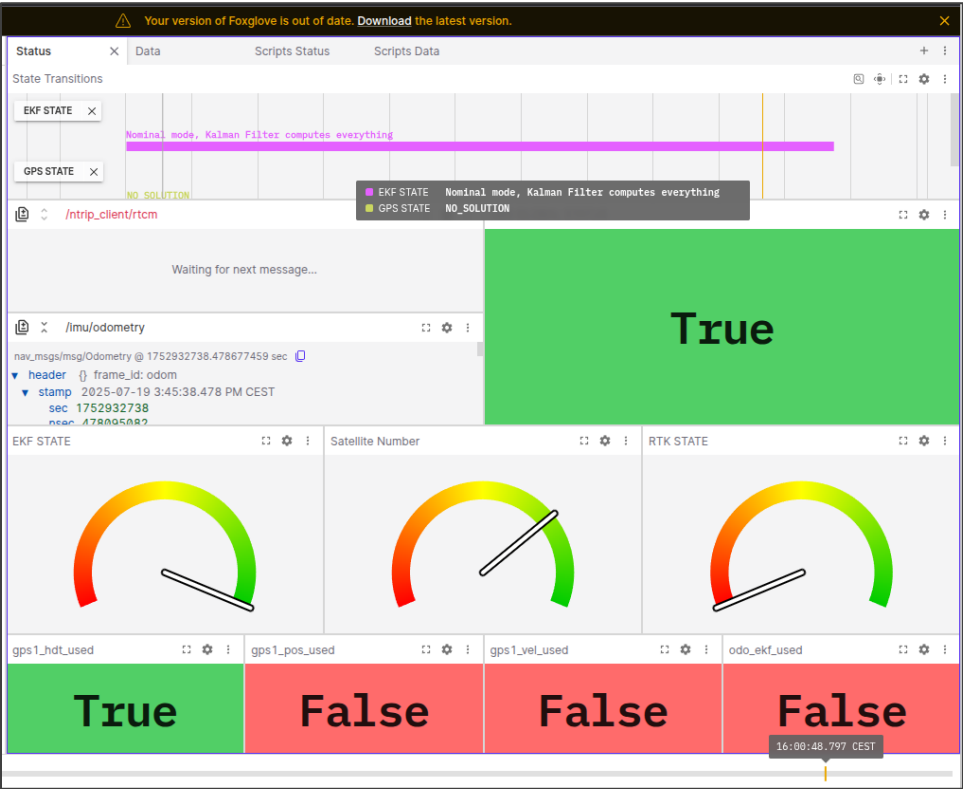
- Finally, we add some additional panel to **publish and visualize the transform tree**.

The screenshot displays the Foxglove Bridge interface with the following components:

- Panel:** Shows the 'Publish panel' configuration. The 'Topic' is set to '/tf' and the 'Message schema' is 'tf2_msgs/msg/TFMessage'. A tooltip shows the 'Editing mode' as 'tf2_msgs/msg/TFMessage'.
- Publish /tf:** Displays the JSON message schema for the 'tf' topic, showing a list of transforms with their headers, frame IDs, and transformations.
- Transform Tree:** Visualizes the transform tree structure. It shows a 'base_link' node connected to four other nodes: 'head_link', 'leg_back_left', 'leg_back_right', and 'leg_front_right'.

Visualization tools (7 / 7)

- You can **do so many things with Foxglove**, even add **script panel** in wich you can create custom in app scripts (**TypeScript**). You can change your data easily to create very easy to visualize layout for complex systems to see immediately if everything works fine on your project.



Layouts from ARECE system.

7 micro-ROS

Micro-ROS introduction (1 / 4)

- Micro-ROS is an **extension of ROS2 designed specifically for resource-constrained embedded systems**. This means it can bring **ROS2 capabilities to microcontrollers** such as ESP32.
- It enables also **seamless integration between embedded devices and ROS2 ecosystem**. This means you can connect and interact within your computer and a microcontroller.
- As we know traditional **ROS2 requires hundreds of MB of RAM** and runs on Linux. As you followed perfectly your microcontroller course, you know that we **only have kilobytes of RAM**.
- Integration into External Tools:

Arduino IDE



The micro-ROS for Arduino support package is a special **bare-metal** provided as a set of **precompiled libraries**.

ESP-IDF



ESPRESSIF

Official dev framework for ESP32. The micro-ROS component allows to **integrate the API in an ESP-IDF project** just by **cloning** or **copying a folder**.

STM32CubeMX



Configured using the static library builder **docker image**. Allows micro-ROS integration into STM32 controller.

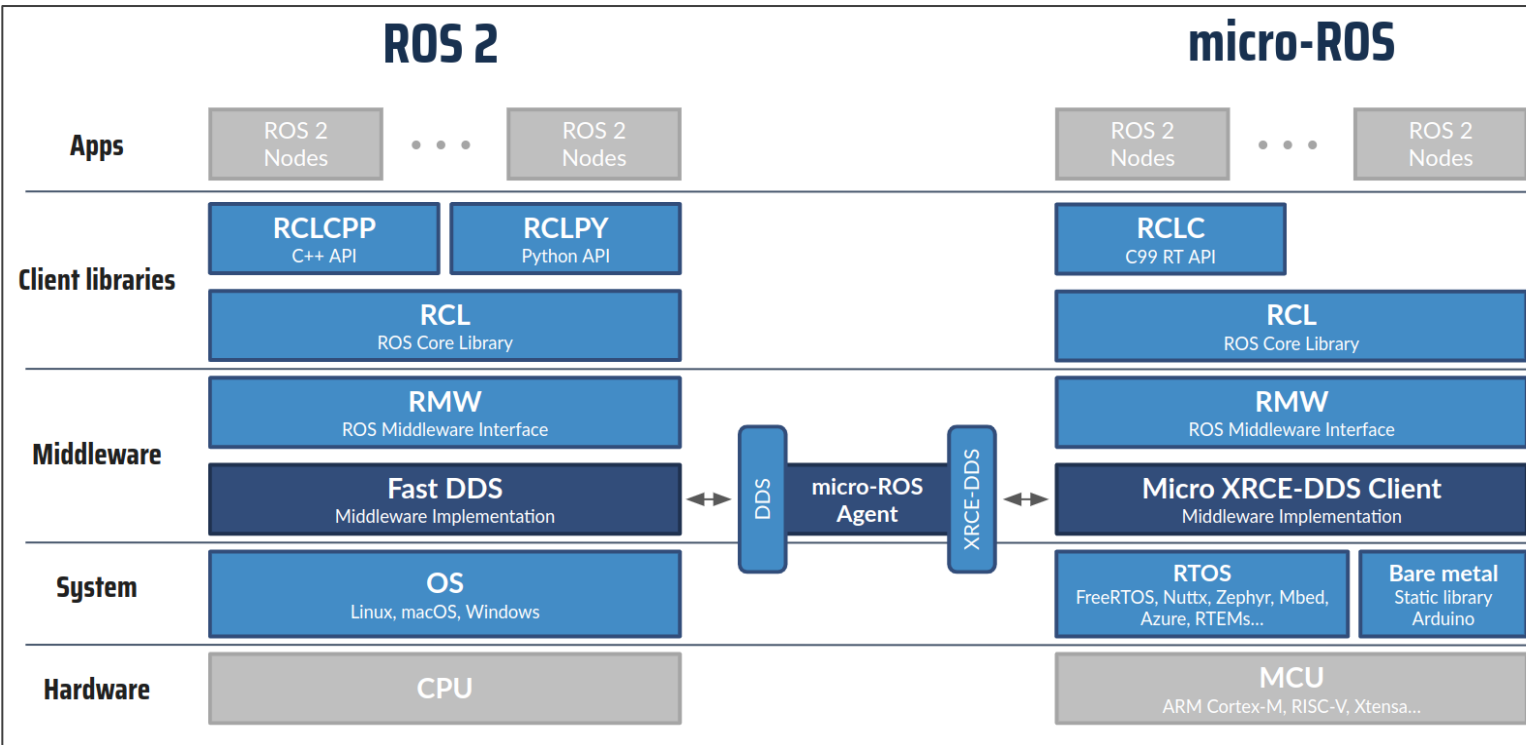


Micro-ROS introduction (2 / 4)

RCLC (ROS Client Library) Optimized C API for MCUs. It is a **C99 implementation** and supports all major ROS 2 concepts.

Micro XRCE-DDS Middleware is a DDS for eXtremely Resource Constrained Environments . Uses a **Client-Server architecture**. The **Client** runs on **MCU** and the **Agent** runs on **ROS2 side**. The state resides in agent and enables sleep cycles for power saving.

RTOS Layer – Provides POSIX interface. Can also run on **bare metal (No OS at all)**



Micro-ROS introduction (3 / 4)

- **Micro-ROS and ROS2 are very similar in very key API concepts**, such as Nodes, Pub/sub, services, actions, etc. This allows to have a full interoperability with ROS2.
- However, there are some differences in the implementation and the interface you work between the two.

	ROS2	Micro-ROS
OS	Linux	RTOS (FreeRTOS / Zephyr / NuttX)
Language	C++ / Python	C 99
Memory	Hundreds of MB	Kilobytes
Middleware	DDS (Fast-DDS, CycloneDDS)	Micro XRCE-DDS
Allocation	Dynamic memory allocation	Static (zero dynamic at runtime)
API	RCLPP and RCLPY API introduced earlier	RCL and RCLC API

Micro-ROS introduction (4 / 4)

- Key features of micro-ROS:

Supported RTOS

FreeRTOS (most popular)
Zephyr (modern architecture)
NuttX (Similar to Unix)
Bare metal possible (limited features)

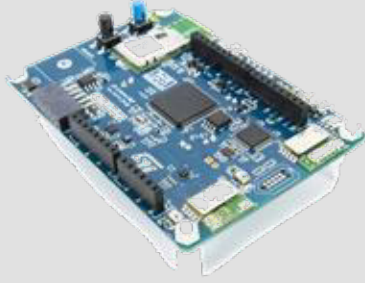
Real-Time Capabilities

Deterministic behavior
User control over timing
Custom execution management
Priority-based callback scheduling

Client-Agent Communication

Works with UART, USB, UDP, TCP,
WiFi, Bluetooth and 6LoWPAN
Easy to implement a custom
transport

- Supported Hardware: Minimum requirements (32-bit architecture, at least 32 kB RAM, communication peripherals (UART, USB, WiFi etc)).

ESP32	Arduino Portenta	Raspberry Pi Pico	STM32L4 Discovery	Arduino Due
				

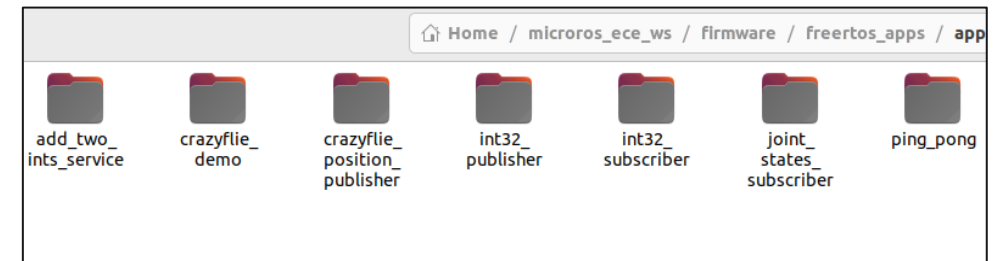
Basic demonstration (1 / 4)

- In this simple demonstration we will create a **publisher node on the ESP32**. For that we will use [micro-ros-setup tool](#), it allows to **build and flash Micro-ROS to every supported platform**.
- First, we create a workspace and download the micro-ROS tools:

```
mkdir microros_ece_ws  
cd microros_ws  
git clone -b $ROS_DISTRO https://github.com/micro-ROS/micro_ros_setup.git src/micro_ros_setup
```

- Then update dependencies using rosdep and build / source the project:

```
sudo apt update && rosdep update  
rosdep install --from-path src --ignore-src -y  
colcon build  
source install/local_setup.bash
```



Other demonstrations available at:
[/firmware/freertos_apps/apps](#)

Basic demonstration (2 / 4)

- The following step will **create firmware workspace** that targets all the required code. It will create folder named *firmware* in the workspace (depending on you rcomputer this can be very long):

```
ros2 run micro_ros_setup create_firmware_ws.sh freertos esp32
```

Micro-Ros component **RTOS** **Target**

- **Configure the created firmware**, we want to show in the example a wifi communication.

- ROS is a wonderful tool but sometimes **the different versions of each tool can raise errors.**

- The requirements is to have ESP-IDF v4.4, however when doing the command: `ls firmware/toolchain/espressif/python_env`

- It will display version 4.1 and this is because we are using ROS humble

- To correct this issue, we must deactivate the wifi_provisioning. Add the following line: `CONFIG_WIFI_PROV_MANAGER_ENABLED=n`

- Begin of file: `firmware/freertos_apps/microros_esp32_extensions/sdkconfig.defaults`

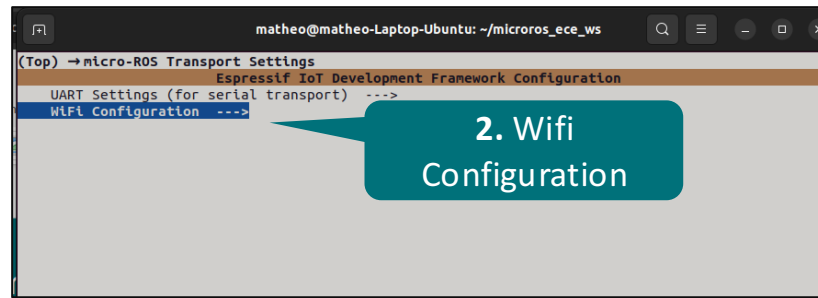
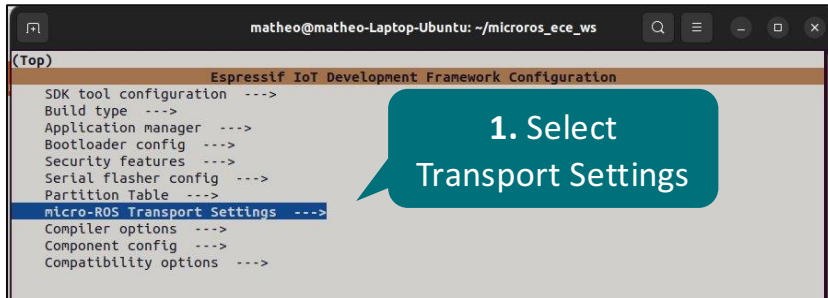
- Finally, we can configure: `ros2 run micro_ros_setup configure_firmware.sh int32_publisher -t udp -I [Computer_IP_ADDRESS] -p 8888`

Execute ifconfig command to get it

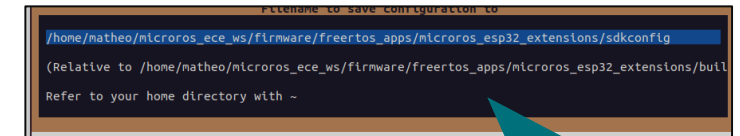
Basic demonstration (3 / 4)

- Now configure the wifi credential to allow the ESP32 to connect to Network:

```
ros2 run micro_ros_setup build_firmware.sh menuconfig
```



- 3. Enter the wifi SSID and password (the one connected on your computer) and press S to save



- Finally, we can build the firmware!

```
ros2 run micro_ros_setup build_firmware.sh
```

- We will now **flash the firmware to ESP32** for that plug with micro-usb the board to your computer.

```
ros2 run micro_ros_setup flash_firmware.sh
```

 (Takes a bit of time at first)

- The micro-ROS is ready to be connected we just need to create and run the **micro-ROS agent that will talk to ROS2 world**.

```
ros2 run micro_ros_setup create_agent_ws.sh
ros2 run micro_ros_setup build_agent.sh
source install/local_setup.bash
```

Basic demonstration (4 / 4)

- The client and the agent are ready, we can run the agent to give access to ROS2 dataspace.

```
ros2 run micro_ros_setup micro_ros_agent udp4 --port 8888
```

- When you run you will get feedback to see topics and ID, you should get results that looks like this:

```
[1657213446.525162] info | TermiosAgentLinux.cpp | init | runn
ng... | fd: 3
[1657213446.525443] info | Root.cpp | set_verbose_level | logger s
etup | verbose_level: 4
[1657213449.884557] info | Root.cpp | create_client | create
 | client_key: 0x3B092B85, session_id: 0x81
[1657213449.884646] info | SessionManager.hpp | establish_session | session
established | client_key: 0x3B092B85, address: 0
[1657213449.912546] info | ProxyClient.cpp | create_participant | particip
ant created | client_key: 0x3B092B85, participant_id: 0x000(1)
[1657213449.929375] info | ProxyClient.cpp | create_topic | topic cr
eated | client_key: 0x3B092B85, topic_id: 0x000(2), participant_id: 0x000(1)
[1657213449.939223] info | ProxyClient.cpp | create_publisher | publishe
r created | client_key: 0x3B092B85, publisher_id: 0x000(3), participant_id: 0x000(
1)
[1657213449.952066] info | ProxyClient.cpp | create_datawriter | datawrit
er created | client_key: 0x3B092B85, datawriter_id: 0x000(5), publisher_id: 0x000(3
)
```

- In the end when we run the command to see the topics we see: `ros2 topic list`

```
/freertos_int32_publisher
/parameter_events
/rosout
```

Magic! We see the topic published by the ESP32 on our computer



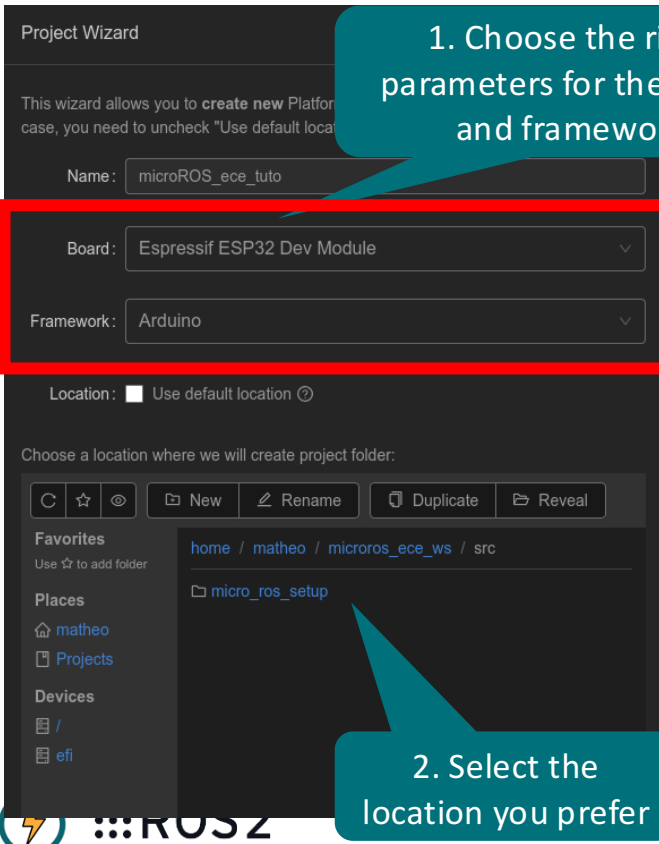
The time between sending and receiving is important in a project. It might be a good start to measure this propagation time!

Create your project with Arduino (1 / 5)

If you are not comfortable with PlatformIO see the tutorial NodeMCU ESP on the toolbox

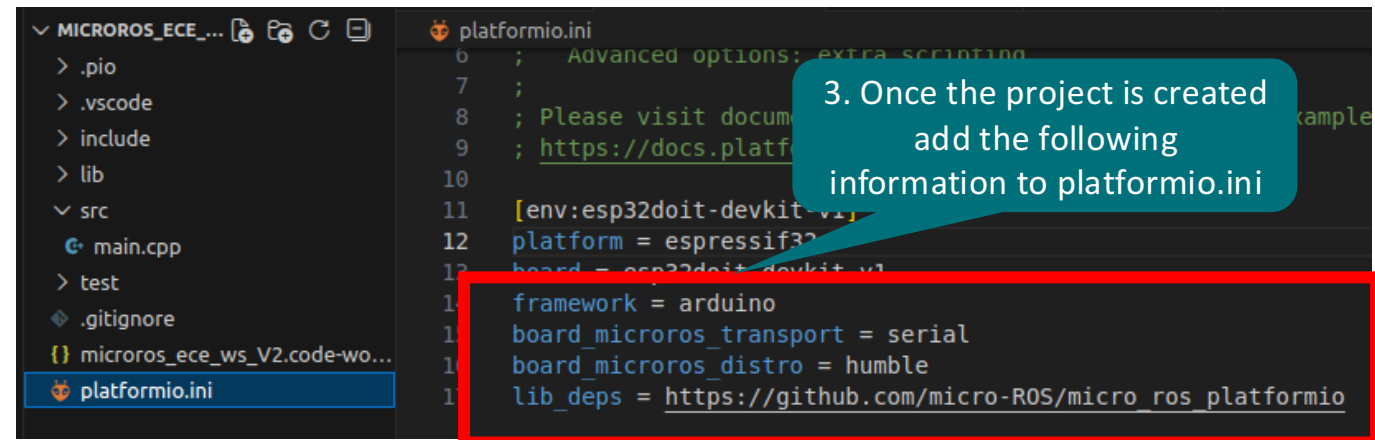
- In this part we will create **our own code for a micro-ROS project** to send sensor data from the ESP32, treat the data in the computer and send the result to the ESP32. This time using **USB serial communication** and using PlatformIO tool to create the project. We will have a project that read an ultrason sensor data then publish it, treated with a ROS2 node and sent the result to the microcontroller.

Create a new project on platformio

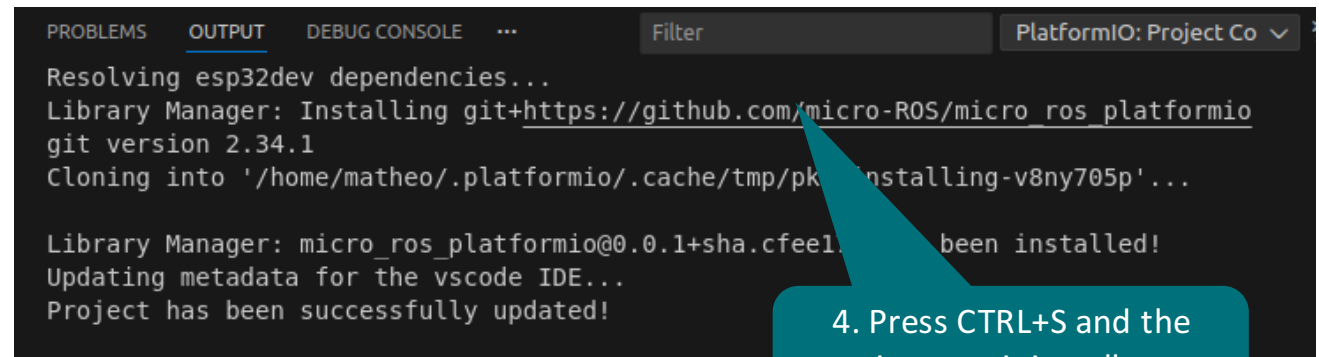


1. Choose the right parameters for the board and framework

2. Select the location you prefer



3. Once the project is created add the following information to platformio.ini



4. Press CTRL+S and the magic starts it install every dependencies we need

Create your project with Arduino (2 / 5)

- In the main.cpp file put the following code (take a look to understand):

Executor, support and allocator provides the capability to have the communication between nodes from our ESP32 and nodes from the personal computer

```
#include <Arduino.h>
#include <micro_ros_platformio.h>
#include <rcl/rcl.h>
#include <rcl/rcl.h>
#include <rcl/executor.h>
#include <std_msgs/msg/float32.h>
#include <std_msgs/msg/string.h>

#define TRIG_PIN 5
#define ECHO_PIN 18

rcl_publisher_t distance_pub;
rcl_subscription_t cmd_sub;
rcl_timer_t timer;
rcl_executor_t executor;
rcl_node_t node;
rcl_allocator_t allocator;
rcl_support_t support;
std_msgs__msg__Float32 distance_msg;
std_msgs__msg__String cmd_msg;
char cmd_buffer[20];

float read_distance_cm() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);
    long duration = pulseIn(ECHO_PIN, HIGH, 30000); // timeout 30 ms
    if (duration == 0) return -1.0;
    return duration * 0.034 / 2.0;
}

void timer_callback(rcl_timer_t *, int64_t) {
    float d = read_distance_cm();
    if (d > 0) {
        distance_msg.data = d;
        rcl_publish(&distance_pub, &distance_msg, NULL);
    }
}

void timer_callback(rcl_timer_t *, int64_t) {
    float d = read_distance_cm();
    if (d > 0) {
        distance_msg.data = d;
        rcl_publish(&distance_pub, &distance_msg, NULL);
    }
}
```

```
void cmd_callback(const void * msgin) {
    const std_msgs__msg__String * msg = (const std_msgs__msg__String *)msgin;
    Serial.print("Commande reçue: ");
    Serial.println(msg->data.data);
}

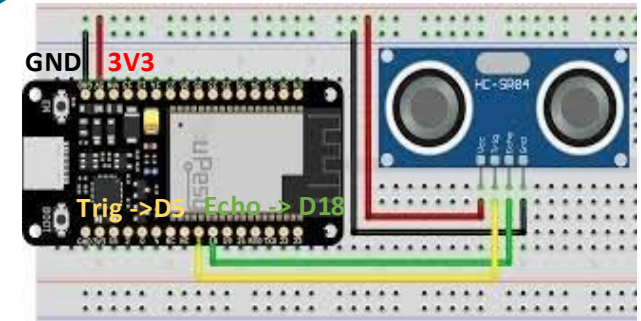
void setup() {
    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);
    Serial.begin(115200);
    set_microros_serial_transports(Serial);
    allocator = rcl_get_default_allocator();
    rcl_support_init(&support, 0, NULL, &allocator);
    rcl_node_init_default(&node, "esp32_ultrasonic", "", &support);
    rcl_publisher_init_default(
        &distance_pub,
        &node,
        ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, Float32),
        "/ultrasonic/distance");
    rcl_subscription_init_default(
        &cmd_sub,
        &node,
        ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, String),
        "/ultrasonic/cmd");
    rcl_timer_init_default(
        &timer,
        &support,
        RCL_MS_TO_NS(200),
        timer_callback);
    cmd_msg.data.data = cmd_buffer;
    cmd_msg.data.capacity = sizeof(cmd_buffer);
    cmd_msg.data.size = 0;
    rcl_executor_init(&executor, &support.context, 2, &allocator);
    rcl_executor_add_timer(&executor, &timer);
    rcl_executor_add_subscription(
        &executor, &cmd_sub, &cmd_msg, &cmd_callback, ON_NEW_DATA);
}

void loop() {
    rcl_executor_spin_some(&executor, RCL_MS_TO_NS(100));
}

}C
```

After writing the code, CTRL+S, then verify and upload using the icons (don't forget to put the board manually on flash mode – see toolbox)

Create your project with Arduino (3 / 5)



Circuit connection diagram

- In this part we will **launch the agent** just as we did in the previous part, but this time **no needs to create a firmware** all the job is already done by platformIO.
- In the workspace from the previous part run the following command :

```
colcon build
source install/setup.bash
ros2 run micro_ros_setup create_agent_ws.sh
ros2 run micro_ros_setup build_agent.sh
ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyUSB0
```

We can already see the different nodes coming from micro ROS

```
matheo@matheo-Laptop-Ubuntu:~/microros_ece_ws$ ros2 topic list
/parameter_events
/rosout
/ultrasonic/cmd
/ultrasonic/distance
```

- When we run the command if everything works fine we get :

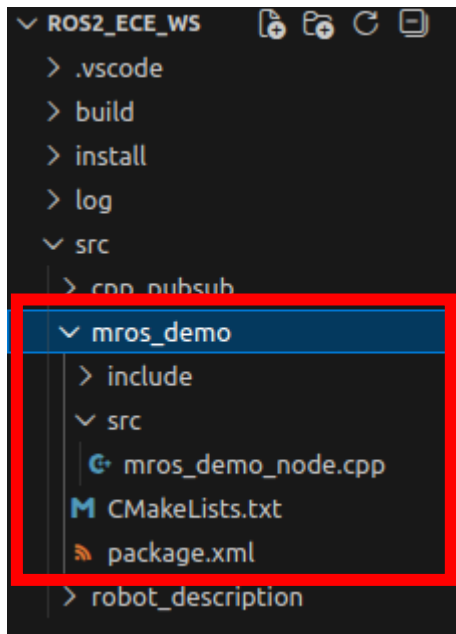
```
matheo@matheo-Laptop-Ubuntu:~/microros_ece_ws$ ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyUSB0
[1767561280.463467] info | TermiosAgentLinux.cpp | init | running... | fd: 3
[1767561280.463860] info | Root.cpp | set_verbose_level | logger setup | verbose_level: 4
[1767561281.138795] info | Root.cpp | create_client | create | client_key: 0x66D6B803, session_id: 0x81
[1767561281.138881] info | SessionManager.hpp | establish_session | session established | client_key: 0x66D6B803, address: 0
[1767561281.158528] info | ProxyClient.cpp | create_participant | participant created | client_key: 0x66D6B803, participant_id: 0x000(1)
[1767561281.173539] info | ProxyClient.cpp | create_topic | topic created | client_key: 0x66D6B803, topic_id: 0x000(2), participant_id: 0x000(1)
[1767561281.182707] info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x66D6B803, publisher_id: 0x000(3), participant_id: 0x000(1)
[1767561281.193472] info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x66D6B803, datawriter_id: 0x000(5), publisher_id: 0x000(3)
[1767561281.207422] info | ProxyClient.cpp | create_topic | topic created | client_key: 0x66D6B803, topic_id: 0x001(2), participant_id: 0x000(1)
[1767561281.216608] info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x66D6B803, subscriber_id: 0x000(4), participant_id: 0x000(1)
[1767561281.227569] info | ProxyClient.cpp | create_datareader | datareader created | client_key: 0x66D6B803, datareader_id: 0x000(6), subscriber_id: 0x000(4)
```


Create your project with Arduino (4 / 5)



This is a training to create your own package, do not forget anything (source, dependencies, etc.)

- We will now create a small package with a node in ROS2 to read and treat the data from the ESP32 (see Part 5 to create a package we will only focus on the code in this part).
- The package for this part and the code will be :



```
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/float32.hpp"
#include "std_msgs/msg/string.hpp"

class UltrasonicProcessor : public rclcpp::Node
{
public:
    UltrasonicProcessor() : Node("ultrasonic_processor")
    {
        sub_ = this->create_subscription<std_msgs::msg::Float32>(
            "/ultrasonic/distance",
            10,
            std::bind(&UltrasonicProcessor::distance_callback, this,
                std::placeholders::_1));

        pub_ = this->create_publisher<std_msgs::msg::String>(
            "/ultrasonic/cmd",
            10);

        RCLCPP_INFO(this->get_logger(), "Ultrasonic processor
        started");
    }
}
```

```
private:
    void distance_callback(const std_msgs::msg::Float32::SharedPtr msg)
    {
        std_msgs::msg::String cmd;
        if (msg->data < 30.0) {
            cmd.data = "gauche";
        } else {
            cmd.data = "droite";
        }
        pub_->publish(cmd);
        RCLCPP_INFO(
            this->get_logger(),
            "Distance: %.1f cm -> %s",
            msg->data,
            cmd.data.c_str());
    }

    rclcpp::Subscription<std_msgs::msg::Float32>::SharedPtr sub_;
    rclcpp::Publisher<std_msgs::msg::String>::SharedPtr pub_;
};

int main(int argc, char ** argv)
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<UltrasonicProcessor>());
    rclcpp::shutdown();
    return 0;
}
```

Create your project with Arduino (5 / 5)

- Finally, to start this node we can run : (make sure to be in the workspace `/ros2_ece_ws` location when running the command)

```
colcon build
source install/setup.bash
ros2 run micro_ros_setup create_agent_ws.sh
ros2 run micro_ros_setup build_agent.sh
ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyUSB0
```

Result from the run of the node :

```
[INFO] [1767562685.106808286] [ultrasonic_processor]: Distance: 6.2 cm -> gauche
[INFO] [1767562685.306728401] [ultrasonic_processor]: Distance: 5.2 cm -> gauche
[INFO] [1767562685.506364798] [ultrasonic_processor]: Distance: 6.2 cm -> gauche
[INFO] [1767562685.706555007] [ultrasonic_processor]: Distance: 8.1 cm -> gauche
[INFO] [1767562685.906860515] [ultrasonic_processor]: Distance: 5.2 cm -> gauche
[INFO] [1767562686.107329123] [ultrasonic_processor]: Distance: 11.6 cm -> gauche
[INFO] [1767562686.311184416] [ultrasonic_processor]: Distance: 78.3 cm -> droite
[INFO] [1767562686.514885668] [ultrasonic_processor]: Distance: 149.5 cm -> droite
[INFO] [1767562686.714604351] [ultrasonic_processor]: Distance: 149.6 cm -> droite
[INFO] [1767562686.914473721] [ultrasonic_processor]: Distance: 149.6 cm -> droite
[INFO] [1767562687.110366642] [ultrasonic_processor]: Distance: 79.7 cm -> droite
[INFO] [1767562687.311006586] [ultrasonic_processor]: Distance: 78.7 cm -> droite
[INFO] [1767562687.510372989] [ultrasonic_processor]: Distance: 79.3 cm -> droite
[INFO] [1767562687.711302041] [ultrasonic_processor]: Distance: 80.1 cm -> droite
[INFO] [1767562687.914589426] [ultrasonic_processor]: Distance: 150.0 cm -> droite
[INFO] [1767562688.114796176] [ultrasonic_processor]: Distance: 149.7 cm -> droite
```

We can visualize the node transmitted to the ESP32

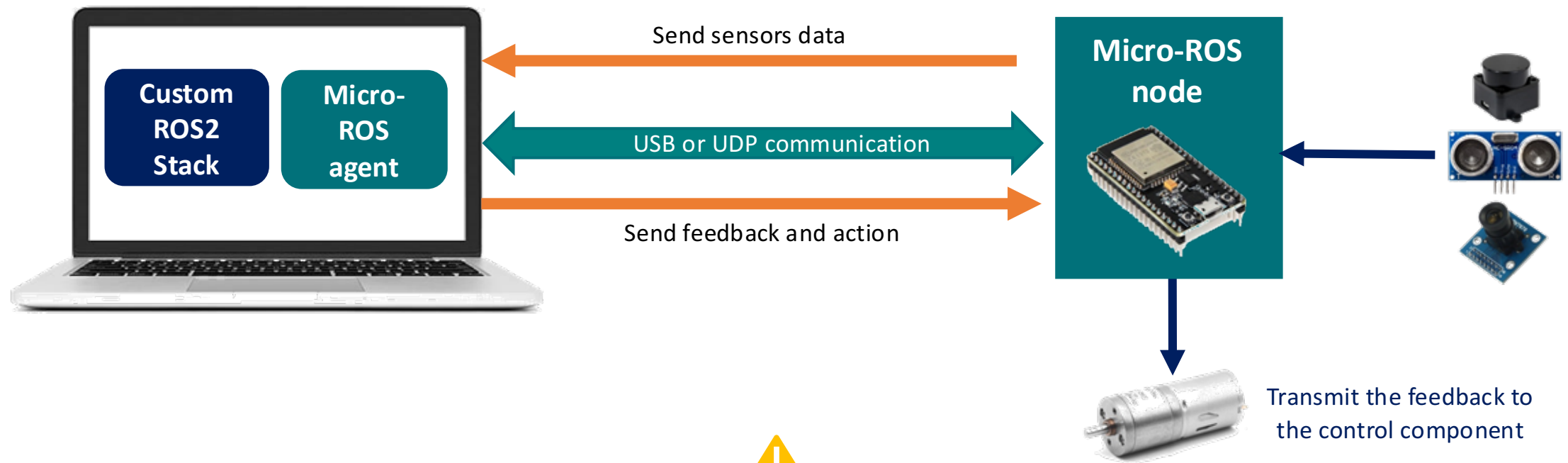
```
matheo@matheo-Laptop-Ubuntu:~/ros2_ece_ws$ ros2 topic echo /ultrasonic/cmd
data: gauche
--
data: gauche
--
data: gauche
--
data: gauche
--
data: gauche
--
data: gauche
--
data: gauche
```

Finally, there are some integrated CLI command in ROS2 for example to see the BW

```
matheo@matheo-Laptop-Ubuntu:~/microros_ece_ws$ ros2 topic bw /ultrasonic/distance
Subscribed to [/ultrasonic/distance]
28 B/s from 3 messages
  Message size mean: 8 B min: 8 B max: 8 B
21 B/s from 5 messages
  Message size mean: 8 B min: 8 B max: 8 B
28 B/s from 10 messages
  Message size mean: 8 B min: 8 B max: 8 B
29 B/s from 14 messages
  Message size mean: 8 B min: 8 B max: 8 B
30 B/s from 18 messages
  Message size mean: 8 B min: 8 B max: 8 B
30 B/s from 22 messages
  Message size mean: 8 B min: 8 B max: 8 B
30 B/s from 26 messages
```

Perspectives

- Micro-ROS offers **great perspective for robotics**. We are now able to have a **small microcontroller that sends sensors data to a larger computer to process and have a full ROS2 stack** that treats the data with large algorithms and actuate or send a feedback to the microcontroller. We can imagine an architecture for an autonomous vehicle:

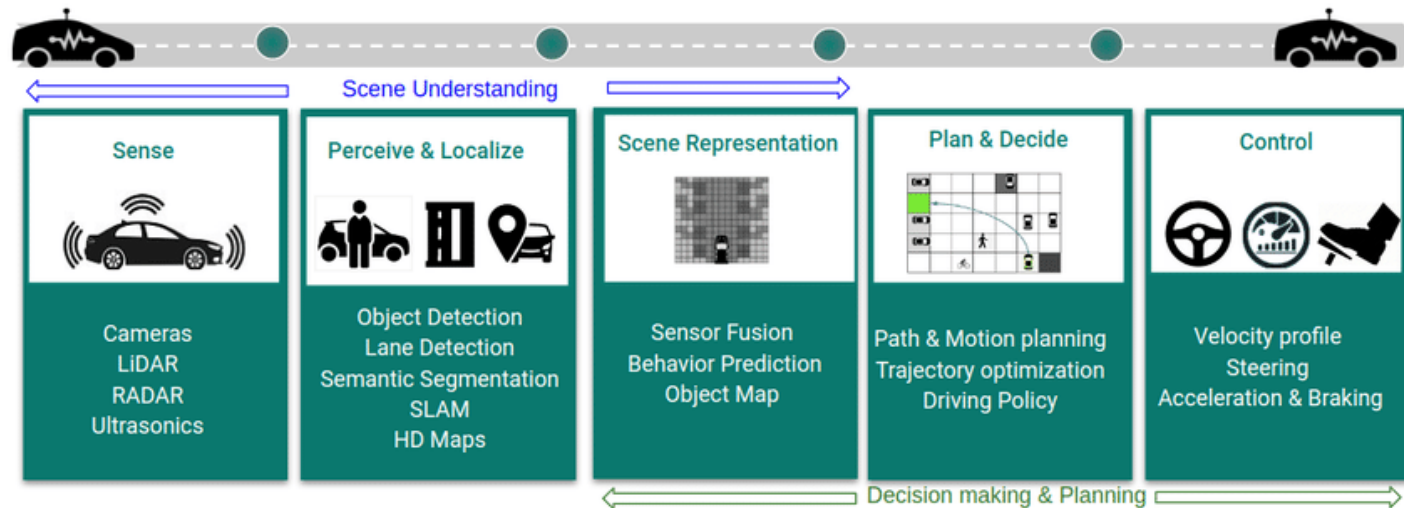


The bandwidth and the speed of transmission becomes very important. Moreover, a sensor can take all the bandwidth making it complicated for other communications.

8 Autonomous vehicles systems

Introduction

- In autonomous vehicle systems, you first **work with many sensors** to understand your environment (obstacles, road, track, etc.). You need multiple sensors because **any single one can fail** due to noise, weather conditions, or other unexpected problems. This is where ROS2 becomes really interesting - it **handles sensor drivers, connects everything together, synchronizes data streams, and manages communication** between components. It simplifies your life so you can **focus on going further with your algorithms for specific topics rather than dealing with low-level integration issues**.

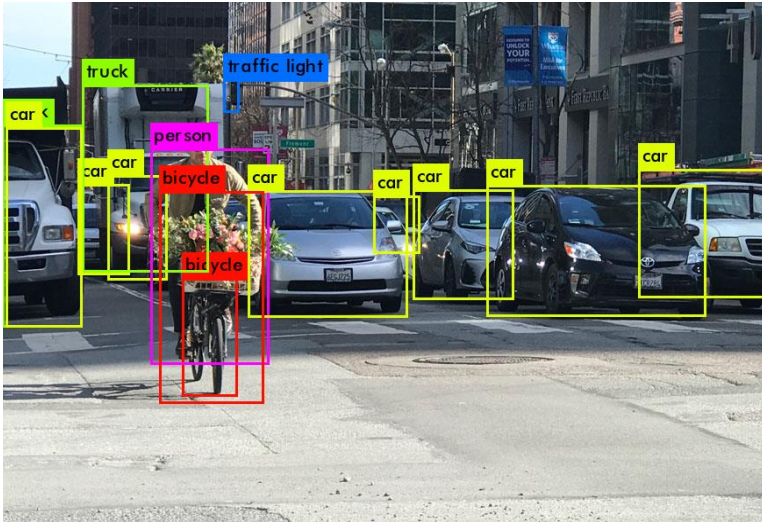


Autonomous vehicle pipeline from
Adithya Ranga and others

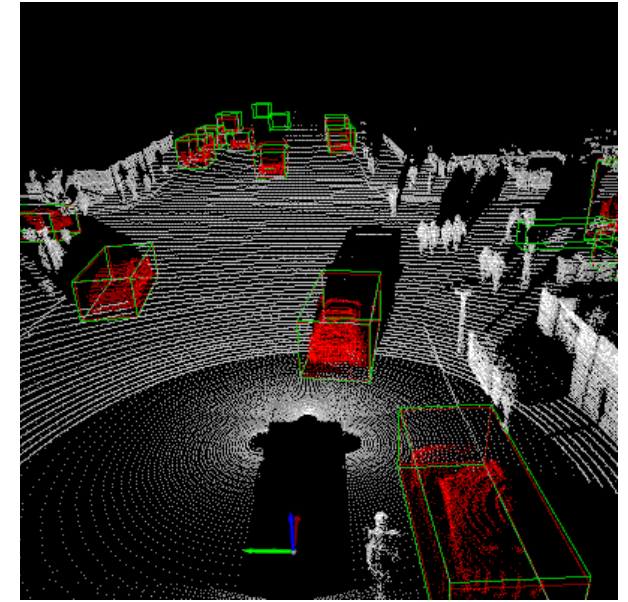
- With ROS2 it is really interesting as each component can be seen as an independent node that can be developed, tested and upgraded separately. If the perception node has a failure, it can be restarted while others can continue their work

Perception

- Perception builds environmental awareness from obstacles, road geometry, etc. The idea in this node is to **fuse data from different sensors that don't "see" the same way or the same things**. For example, cameras see colors and textures while LiDAR will give you the precise distances.



YOLOv7 usage for Autonomous vehicles by Jayita Bhattacharyya

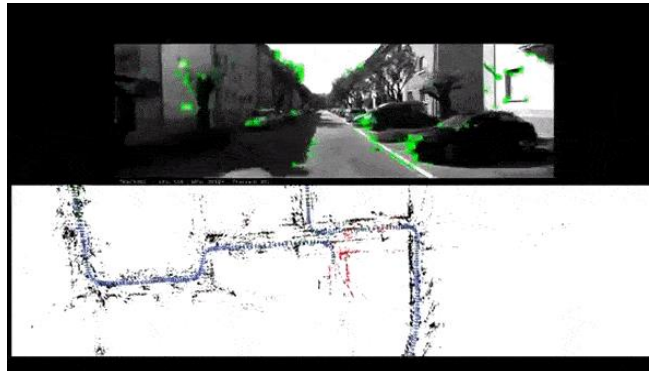


3D LiDAR Object detection

- The **sensor fusion** will allow to build a **dense 3D environmental model** before making decisions called **"mapping"**.
- Currently when working with camera, we do not use anymore classic signal treatment. **State-of-art works are CNNs like YOLO and R-CNN**. However these networks are heavily trained on millions of images, and they need computing power to work.

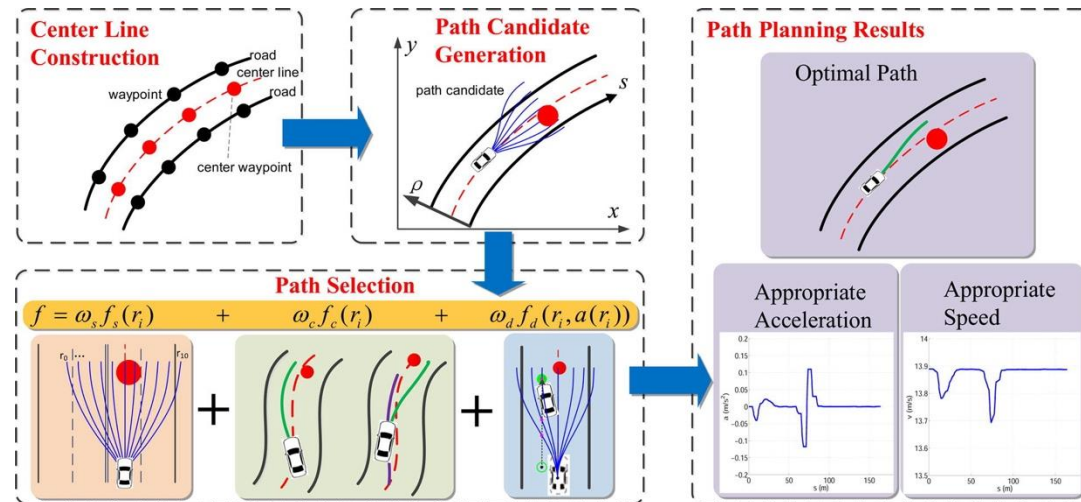
Localisation and SLAM

- **SLAM (Simultaneous localization and mapping)** is about constructing a map of an unknown environment while simultaneously tracking the location of your object (car) within it. Update both the position and the map at all time. It kinds of recreated the world virtually.
- There are two main families of algorithms :
 - Filter-based methods (Extended Kalman Filter for example) : Update the state on-the-go with each new sensor reading (faster but drift over time)
 - Graph based methods (like GraphSLAM) that optimize the entire trajectory using all measurements at once (heavy compute but precise)
- For SLAM we often use LiDAR to have the most precise map construction. There are also many techniques such as ORB SLAM that features camera frames to relocate viewpoints and corrects accumulated drift (**loop closure**)



Planning

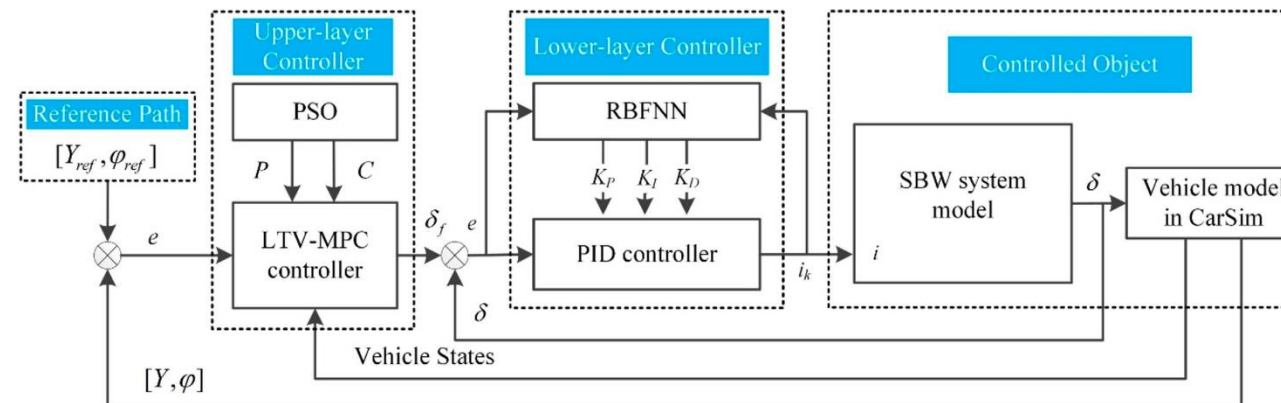
- Now that we have created the map and see perfectly our environment and can be localized within it we want to plan the action or the direction we want to go. For that **global planning** creates a **collision-free path using the full map**. While **local planning** dynamically adjusts based on real time obstacles.
- They exist many algorithms such as Dijkstra or other heuristic but nowadays we use AI model to optimize the path combined with a classic path planner. Sampling based are often use for complex spaces such as **RRT and RRT***, they **randomly sample the space and grow a tree toward the goal**, adapting well to changing environments.



Dynamic path planning from Xuemin Hu and others

Control

- Control is all about making sure the **instruction sent to the actuator is the right one and well translated**. For that we use:
 - PID (Proportional-Integral-Derivative)** control, it is widely used but struggles with complex constraints and nonlinear dynamics. It reacts to current error without predicting the future.
 - MPC (Model Predictive Control)** it optimizes actuator signals over a prediction horizon, considering current state, system dynamics and future trajectory. Longer horizons provide control flexibility but increase computation time. **Achieve perf of kinematic models with half the computation time!**
 - For moderate speed we can use simpler controller such as Ackermann that just follows the path constantly aiming toward a lookahead point with a PID included.



Complex two-layer controller from Zhiwei He and others

9 Advanced resources

Conclusion

- **Take time to understand the code and never reinvent the wheel.** Read existing packages (Nav2, slam_toolbox), understand how they work and customize what you need. **Don't copy-paste blindly.**
- Think like a **system architect**. You don't code only one algorithm when you do ROS, it's about **designing how every node will work together**. The integration and communication between modules is often the hardest part (simplified by ROS2).
- **The embedded engineer's reality!** You work with hardware (sensors, motors, etc.), software and real time constraints all at once. Getting everything synchronized and working reliably, is the real challenge.
- It is important if you want to work further in robotics or domains linked that you **get comfortable with Linux**, it will get easier and easier to debug and monitor your system as long as you practice and take the time to think.
- **Practice with real projects!**

CLI Cheat Sheet (1 / 3)

Package and workspace commands

- **General format of ROS2 CLI** - 'ros2' is the unique entry point for the CLI then :

```
ros2 [command] [verb] <positional-argument> <optional-arguments>
```

For help on ROS2 CLI command:

```
ros2 [command] --help  
ros2 [command] [verb] -h
```

Setup & Environment

```
# Source ROS2 (add to ~/.bashrc for permanent)  
source /opt/ros/humble/setup.bash  
  
# Source workspace  
source ~/ros2_ws/install/setup.bash  
  
# Check ROS2 installation  
printenv | grep -i ROS  
  
# Set domain ID (0-101, isolates ROS2 networks)  
export ROS_DOMAIN_ID=0
```

```
# Create workspace  
mkdir -p ~/ros2_ws/src  
cd ~/ros2_ws  
colcon build  
  
# Create package (Python)  
cd ~/ros2_ws/src  
ros2 pkg create --build-type ament_python <package_name>  
  
# Create package (C++)  
ros2 pkg create --build-type ament_cmake <package_name>  
  
# Create package with dependencies  
ros2 pkg create --build-type ament_python <package_name> --dependencies  
rclpy std_msgs  
  
# List all packages  
ros2 pkg list  
  
# Find package path  
ros2 pkg prefix <package_name>  
  
# Build workspace  
cd ~/ros2_ws  
colcon build  
  
# Build specific package  
colcon build --packages-select <package_name>  
  
# Build with symlink (for Python, no rebuild needed)  
colcon build --symlink-install
```

CLI Cheat Sheet (2 / 3)

Node Commands

```
# List all running nodes
ros2 node list

# Get info about a specific node
ros2 node info /node_name

# Run a node from a package
ros2 run <package_name> <executable_name>
```

Action commands

```
# List all actions
ros2 action list

# List actions with types
ros2 action list -t

# Show action info
ros2 action info /action_name

# Send action goal
ros2 action send_goal /action_name
<action_type> "goal"

# Send goal with feedback
ros2 action send_goal -f /action_name
<action_type> "goal"
```

Topic Commands

```
# List all topics
ros2 topic list

# List topics with message types
ros2 topic list -t

# Show message being published on topic
ros2 topic echo /topic_name

# Show topic info (publishers, subscribers, type)
ros2 topic info /topic_name

# Show topic publishing rate
ros2 topic hz /topic_name

# Publish to a topic (once)
ros2 topic pub /topic_name <msg_type> "data"

# Publish continuously at 1 Hz
ros2 topic pub -r 1 /topic_name <msg_type> "data"
```

Message commands

```
# List all message types
ros2 interface list

# Show message structure
ros2 interface show <msg_type>

# Example: Show Twist message structure
ros2 interface show
geometry_msgs/msg/Twist

# Show message package
ros2 interface package <package_name>
```

Service Commands

```
# List all services
ros2 service list

# List services with types
ros2 service list -t

# Show service type
ros2 service type /service_name

# Call a service
ros2 service call /service_name
<service_type> "request"

# Example: Clear turtlesim background
ros2 service call /clear
std_srvs/srv/Empty
```

CLI Cheat Sheet (3 / 3)

Parameter Commands

```
# List parameters of a node
ros2 param list /node_name

# Get parameter value
ros2 param get /node_name parameter_name

# Set parameter value
ros2 param set /node_name parameter_name value

# Load parameters from file
ros2 param load /node_name params.yaml
```

Launch Files

```
# Launch a launch file
ros2 launch <package_name> <launch_file>

# Example
ros2 launch turtlesim multisim.launch.py

# Launch with arguments
ros2 launch <package_name> <launch_file> arg:=value
```

BAG (Record / Playback)

```
# Record all topics
ros2 bag record -a

# Record specific topics (generally we
prepare this in a launch or config file
ros2 bag record /topic1 /topic2

# Record with custom name
ros2 bag record -o my_bag /topic_name

# Show bag info
ros2 bag info <bag_file>

# Play bag
ros2 bag play <bag_file>

# Play at different speed (2x)
ros2 bag play -r 2 <bag_file>

# Play in loop
ros2 bag play -l <bag_file>
```

Transform (TF) Commands

```
# View TF tree
ros2 run tf2_tools view_frames

# Echo transform between frames
ros2 run tf2_ros tf2_echo <source_frame>
<target_frame>

# Monitor TF
ros2 run tf2_ros tf2_monitor
```

Debugging and Diagnostics (important)

```
# Check if daemon is running
ros2 daemon status

# Start daemon
ros2 daemon start

# Stop daemon
ros2 daemon stop

# Show ROS2 doctor report
ros2 doctor

# View computational graph
rqt_graph

# Monitor topic statistics
ros2 topic hz /topic_name
ros2 topic bw /topic_name # bandwidth

# Kill all ROS2 nodes
killall -9 ros2
```

Bibliography (1 / 2)

- [1] Husarion, « ROS 2 Tutorials - 7. Transformation », Husarion, [s.d.] [online] <https://husarion.com/tutorials/ros2-tutorials/7-transformation/>
- [2] W. WOODALL, « ROS Middleware Interface », ROS 2 Design, 2016 [online] https://design.ros2.org/articles/ros_middleware_interface.html
- [3] The Construct, « ROS2 Developers Guide », The Construct, 2021 [online] <https://www.theconstruct.ai/wp-content/uploads/2021/06/ROS2-Developers-Guide-2021.pdf>
- [4] NCSU, « ROS2 Robot Programming WS », North Carolina State University, 2023 [online] <https://livingston.wordpress.ncsu.edu/files/2023/03/ROS2-Robot-Programming-WS-001.pdf>
- [5] O. STASSE, « ANF 2RM ROS2 », LAAS-CNRS, 2022 [online] <https://homepages.laas.fr/ostasse/Teaching/ROS/2022/anf-2rm-ros2-2022-polycopie.pdf>
- [6] R. FREZZA, « Introduction to ROS 2 », CentraleSupélec, [s.d.] [online] https://frezza.pages.centralesupelec.fr/st5-drones//cours/Introduction_to_ROS2.pdf
- [7] H. KUTLUCA, « Robot Operating System 2 (ROS 2) Architecture », Medium, 2020 [online] <https://medium.com/software-architecture-foundations/robot-operating-system-2-ros-2-architecture-731ef1867776>

Bibliography (2 / 2)

- [8] R. RAMACHANDRAN, « The Complete Beginner's Guide to Using Docker for ROS 2 Deployment », RobotAir, 2025 [online] <https://blog.robotair.io/the-complete-beginners-guide-to-using-docker-for-ros-2-deployment-2025-edition-0f259ca8b378>
- [9] LearnOpenCV, « Introduction to Robot Operating System », LearnOpenCV, [s.d.] [online] <https://learnopencv.com/robot-operating-system-introduction/>
- [10] Articulated Robotics, « URDF », Articulated Robotics, [s.d.] [online] <https://articulatedrobotics.xyz/tutorials/ready-for-ros/urdf/>
- [11] Black Coffee Robotics, « ROS 1 to ROS 2 Migration », Black Coffee Robotics, [s.d.] [online] <https://www.blackcoffeerobotics.com/blog/ros-1-to-ros-2-migration>
- [12] P. LAGUILLAUMIE, « Software development for reactive multi-axis control », Université de Poitiers, 2023 [online] https://theses.hal.science/tel-04409449v1/file/2023_laguillaumie_developpement_logiciel.pdf
- [13] micro-ROS, « micro-ROS », micro-ROS, [s.d.] [online] <https://micro.ros.org/>
- [14] Open Robotics, « ROS 2 Humble Hawksbill Documentation », ROS.org, 2022 [online] <https://docs.ros.org/en/humble/index.html>